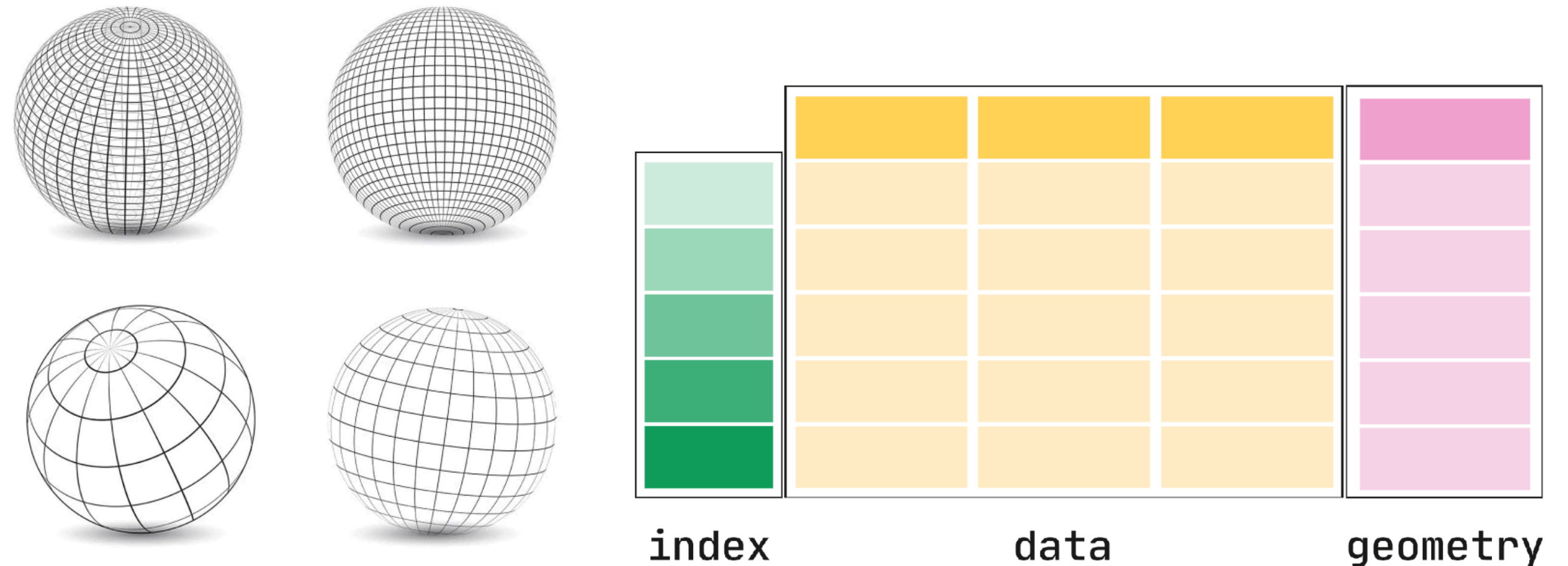


Lecture 2: Geospatial data in Python

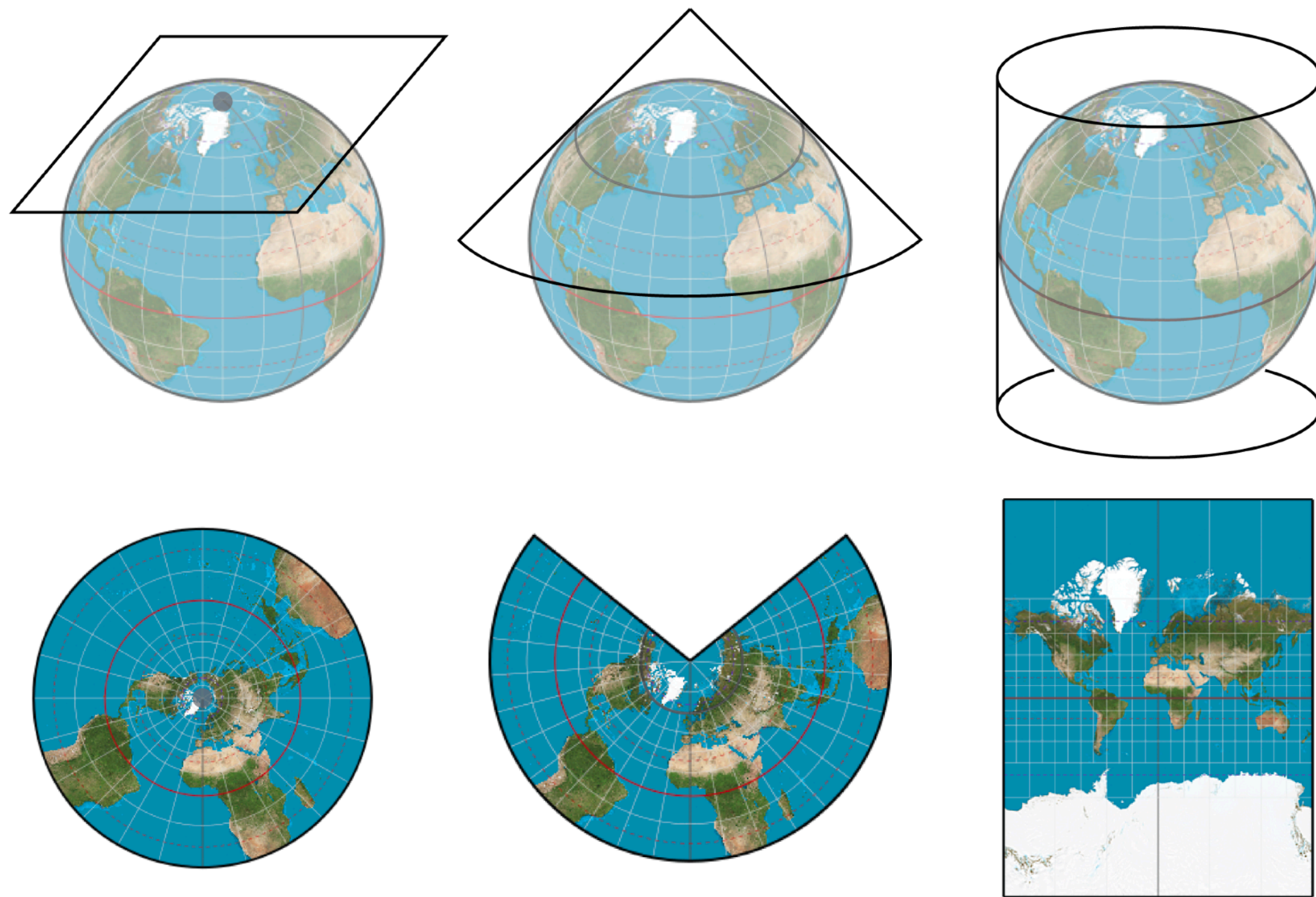
Instructor: Michael Szell

Feb 4, 2026



Today you will learn about....

Coordinate reference systems



GeoPandas/ How to work with vector data



Exam materials are now on learnit

 Project instructions

▼  Old example submissions

 [gds_examplesubmission01.pdf](#)

 [gds_examplesubmission02.pdf](#)

 [gds_examplesubmission03.pdf](#)

 Propose your project here

 Available from 18 March 2026

▼  Project report template

 [gds_report_template_latex.pdf](#)

 [gds_report_template_latex.zip](#)

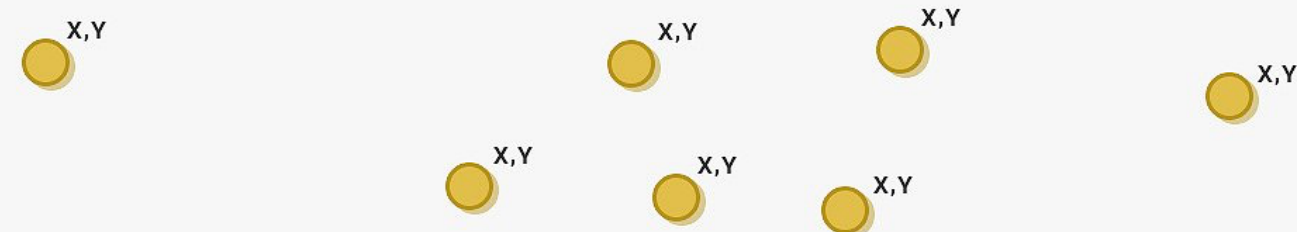
Data sets

There are a lot of open and free spatial data sets, so listed below is only a small list of examples. Always remember to be critical of data quality and origin!

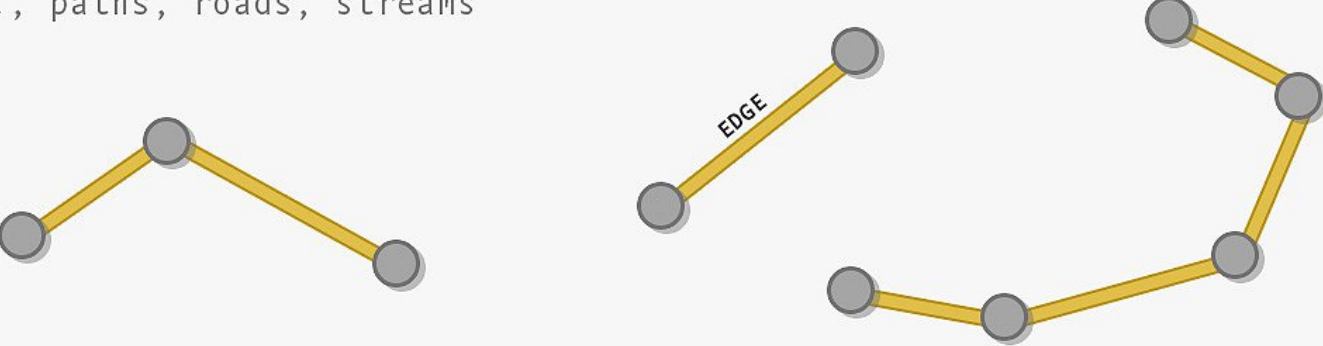
Recap from last time

Geometric objects

Point (node, vertex)
INDIVIDUAL X,Y LOCATIONS
E.g., label, manhole, tower locations



Line (chain)
2 OR MORE POINTS THAT ARE CONNECTED*
E.g., paths, roads, streams



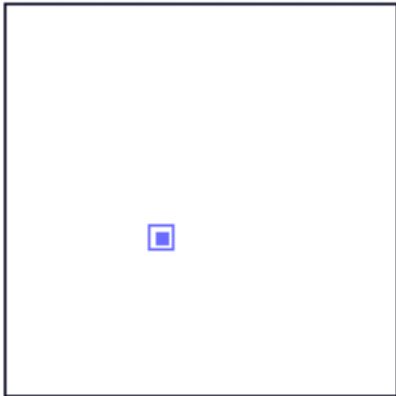
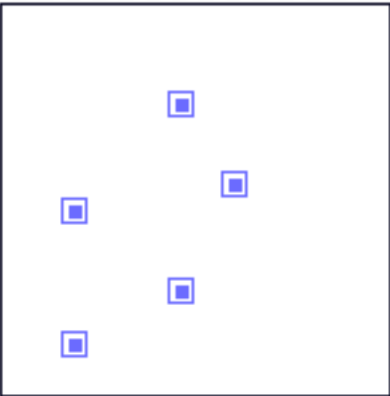
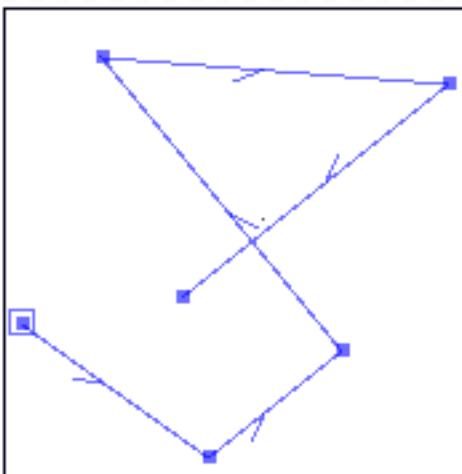
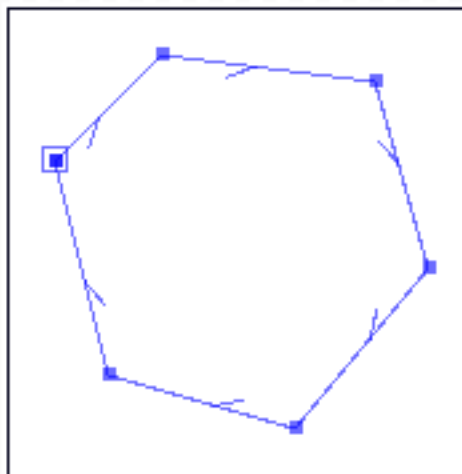
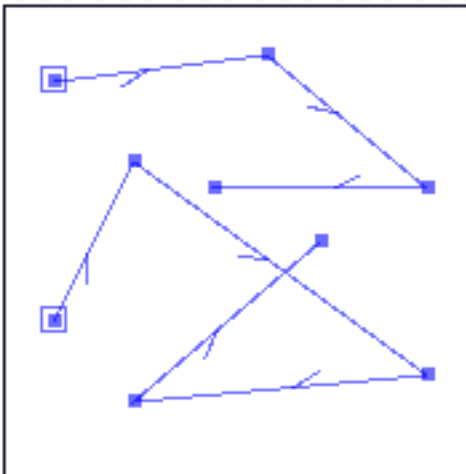
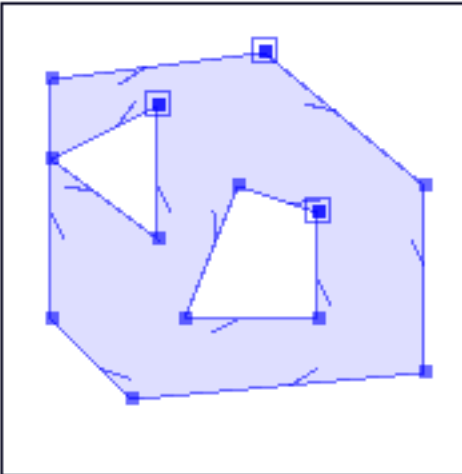
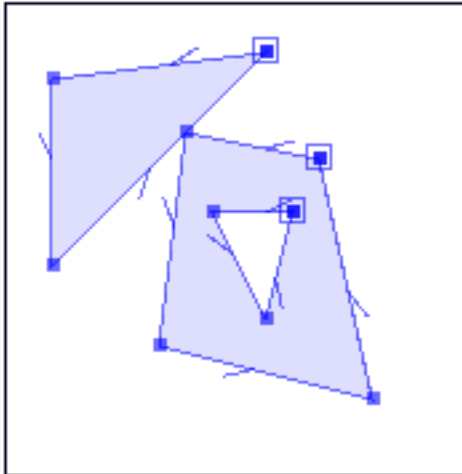
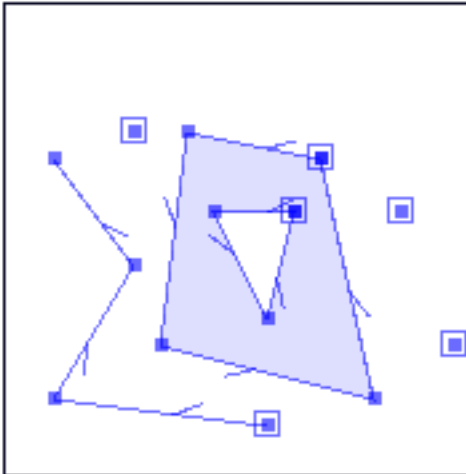
Polygon (area segment)
3 OR MORE VERTICES THAT ARE CONNECTED AND CLOSED
E.g., Land/water boundaries, buildings



* 2 connected points also known as edge or arc.

<https://www.learn datasci.com>

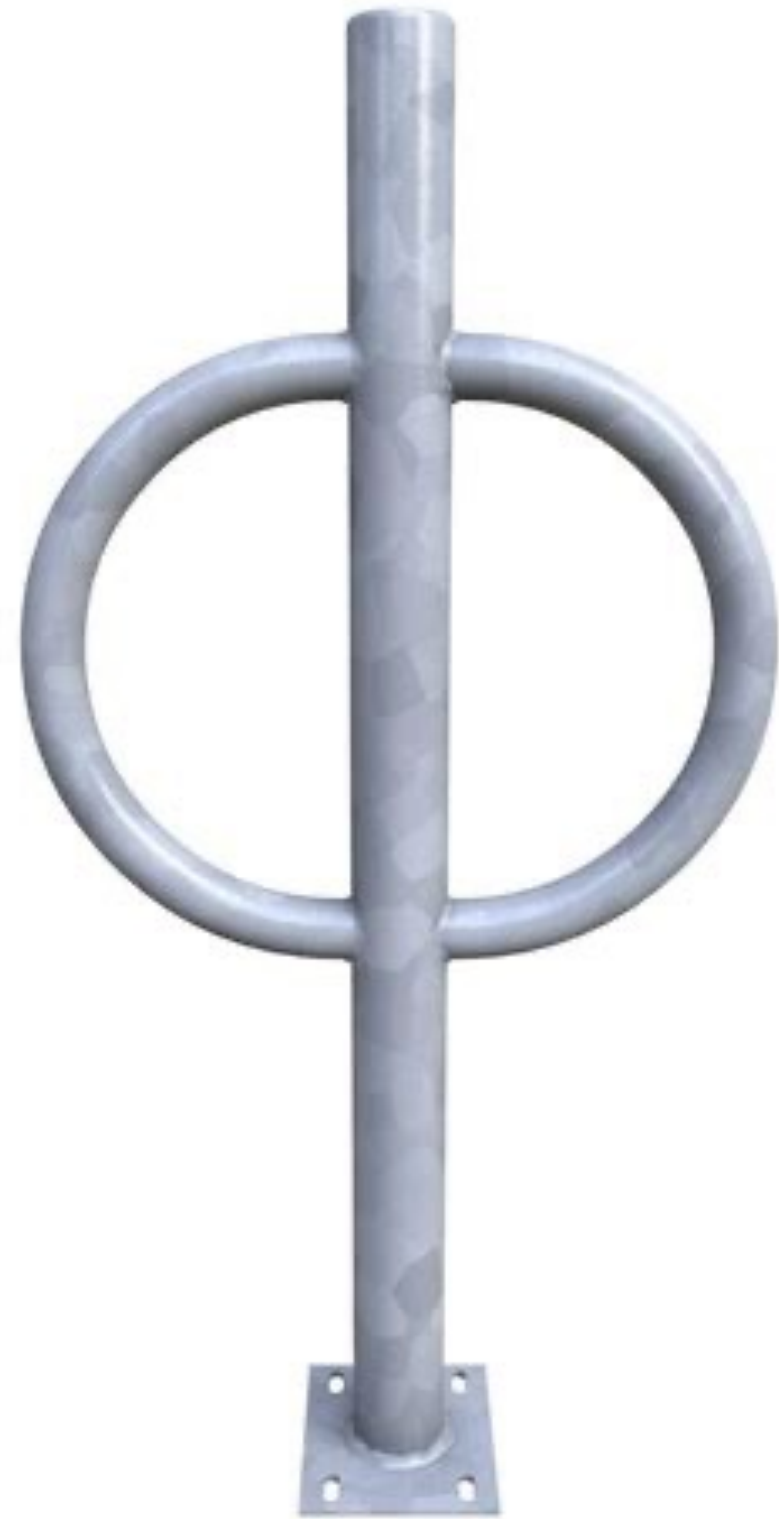
Shapely

		
Point	MultiPoint	
		
LineString	LinearRing	MultiLineString
		
Polygon	MultiPolygon	GeometryCollection

Coordinate tuples



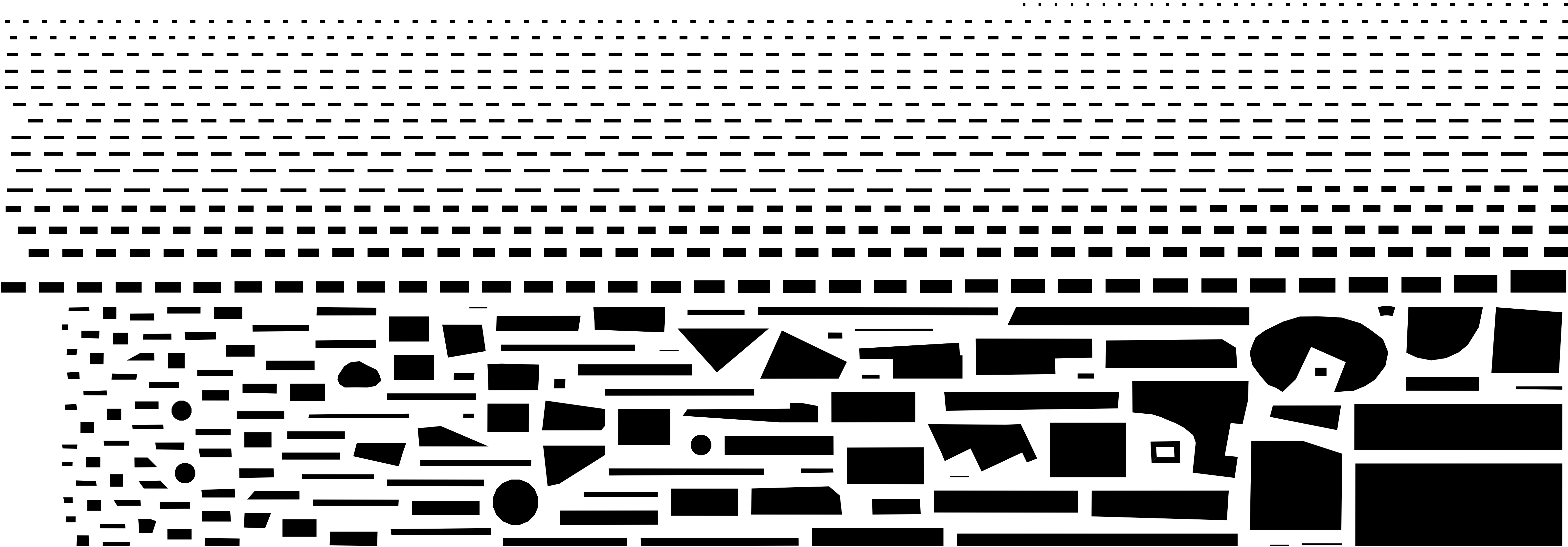
Encoding objects as nodes is often obvious..



..until it isn't



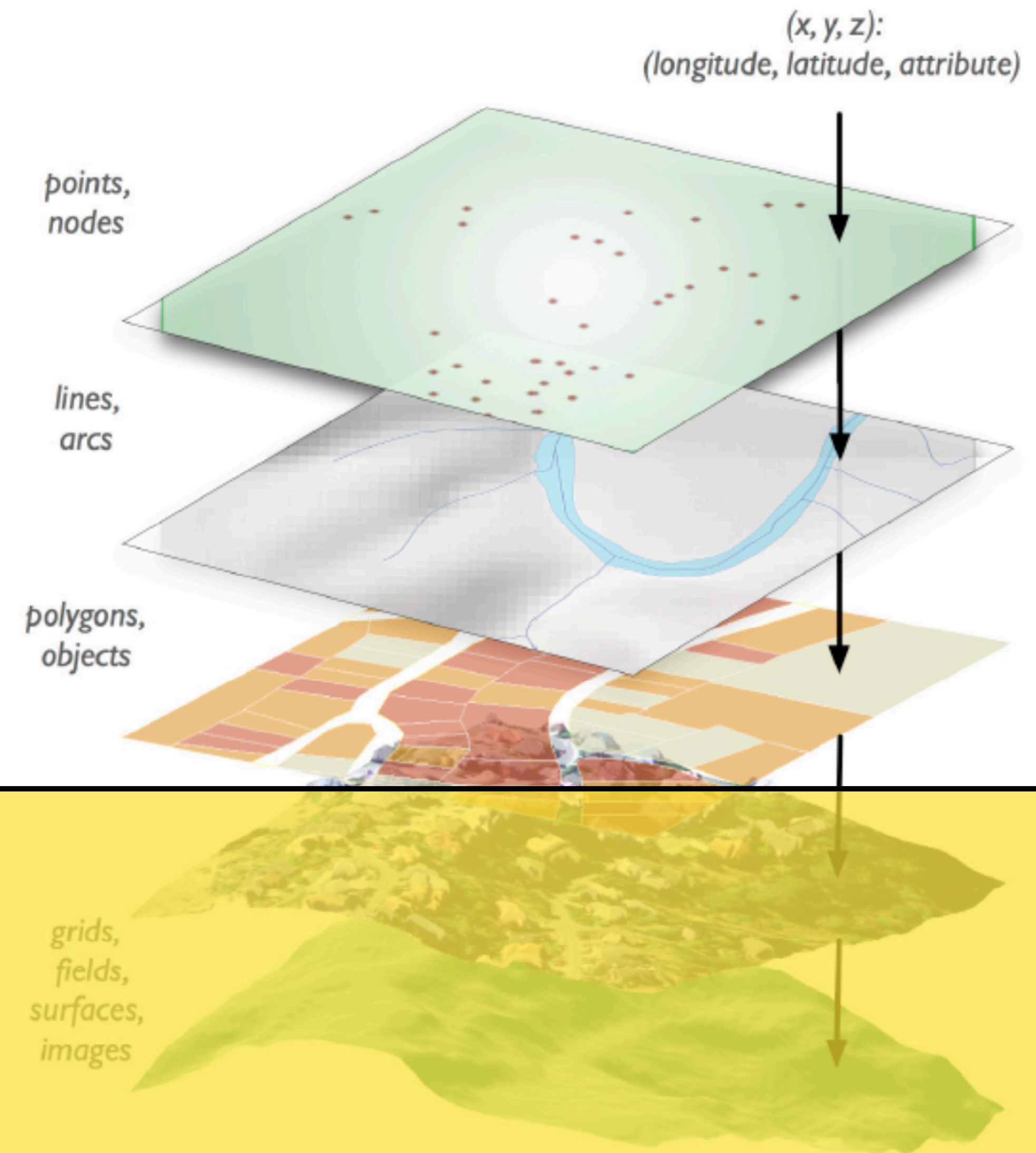
..until it isn't



There is **vector** and **raster** data

Vector: Geometric objects
.gpkg, .shp, .svg, geojson

Raster: Grid of pixels
.tif, .jpg, .png, .bmp



File formats matter (sometimes)

Shapefiles - old classic



File formats matter (sometimes)

Geopackage - 'universal'

GeoJSON - web-optimized

CSV

Shapefiles - old classic

File formats matter (sometimes)

Geopackage - 'universal'

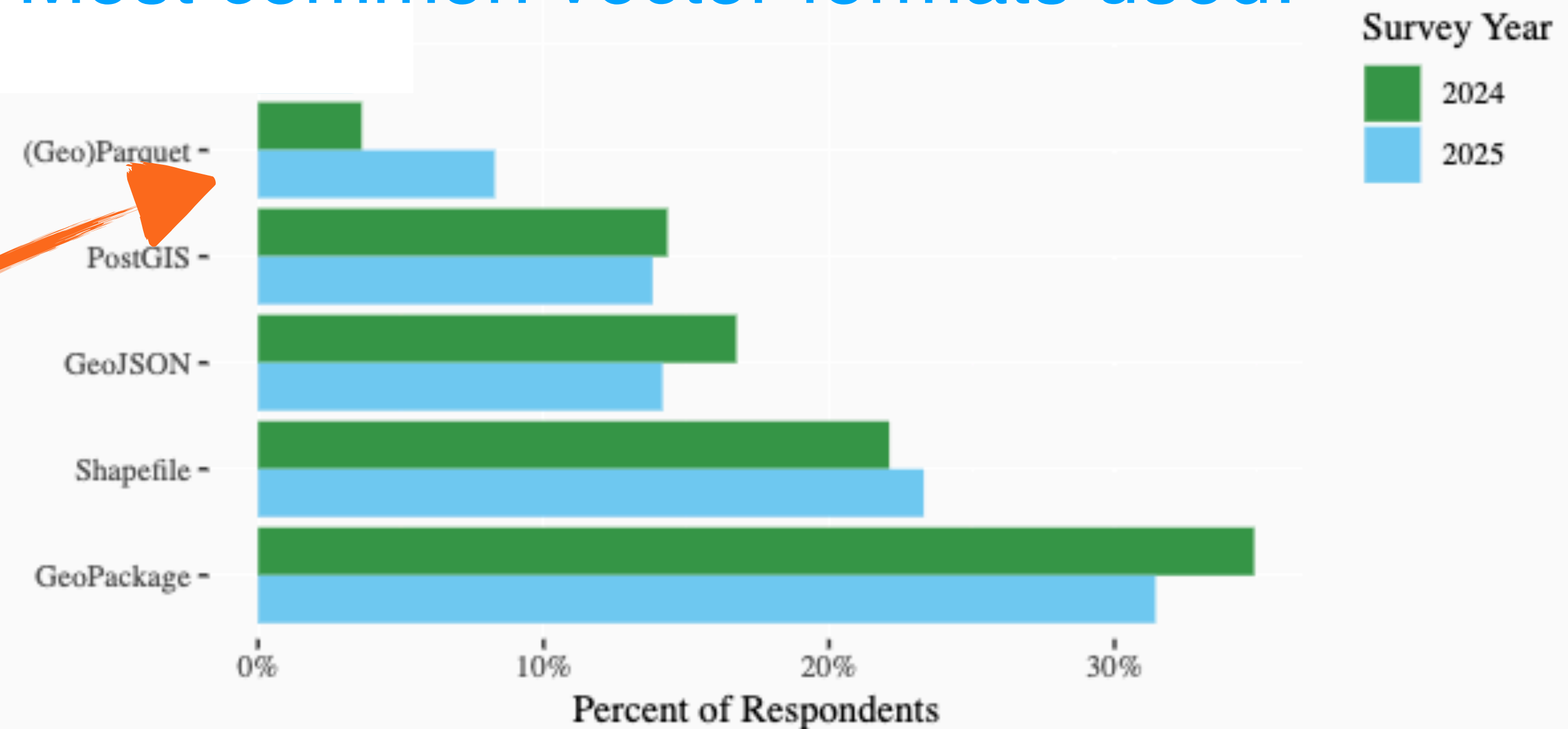
GeoJSON - web-optimized

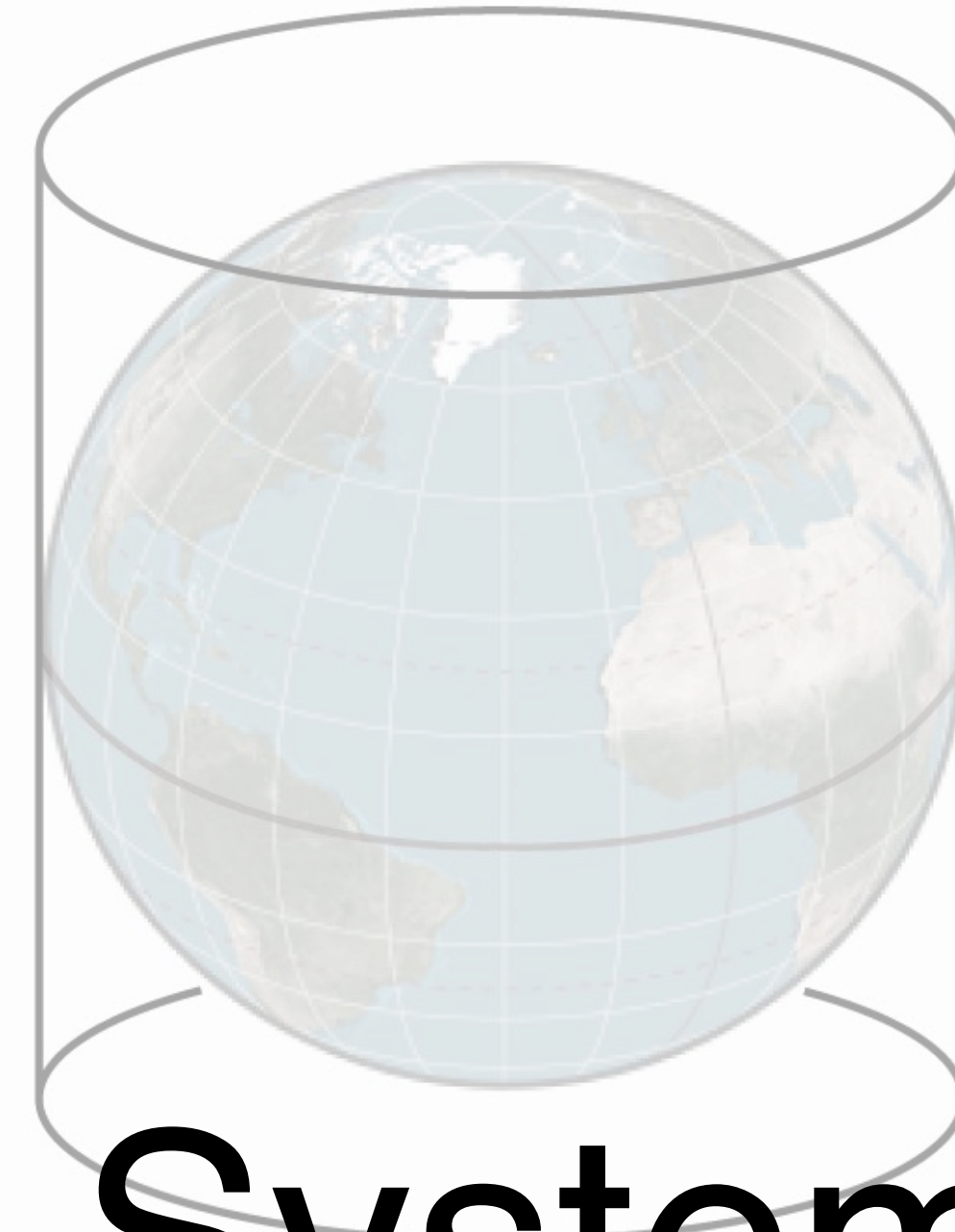
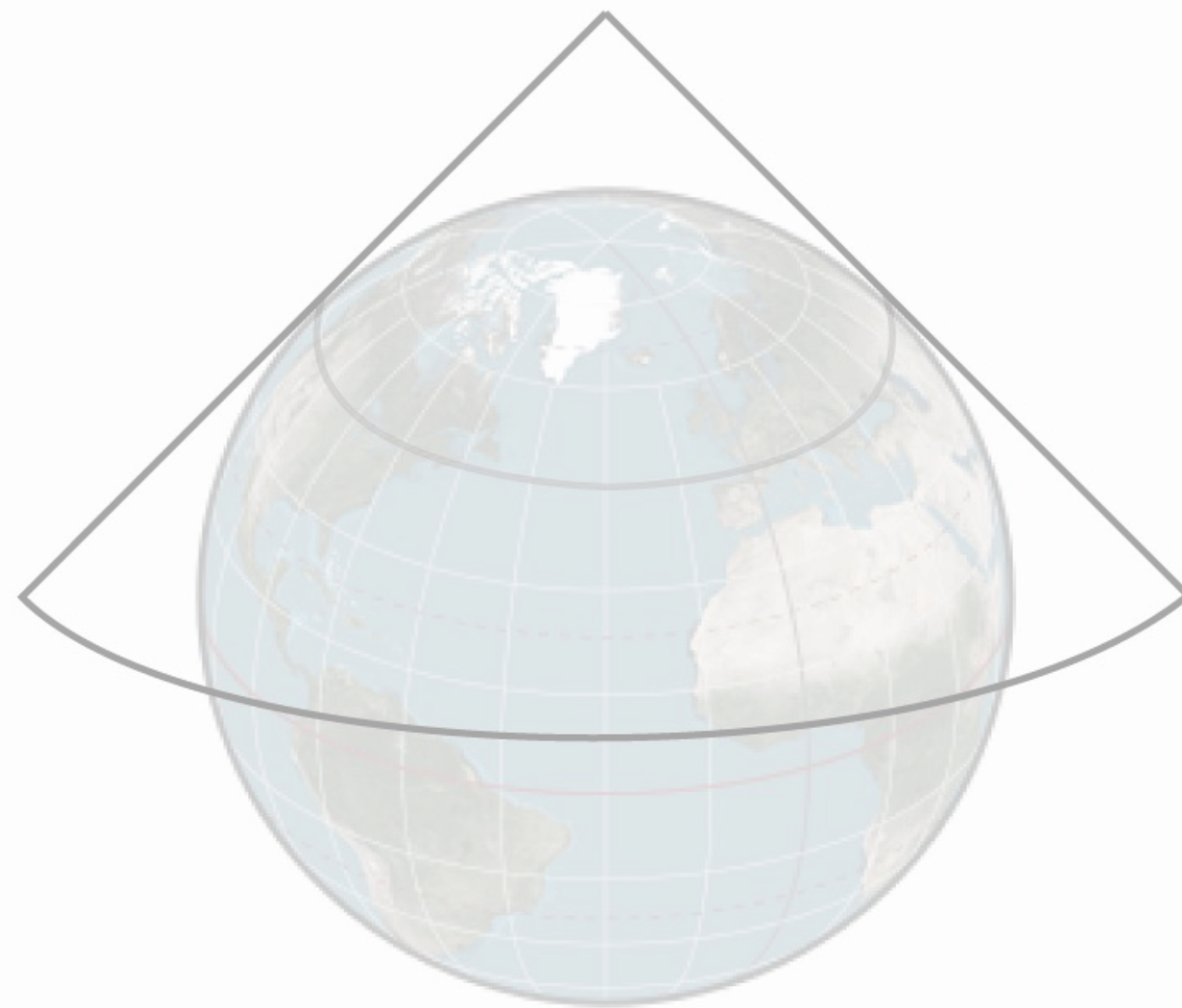
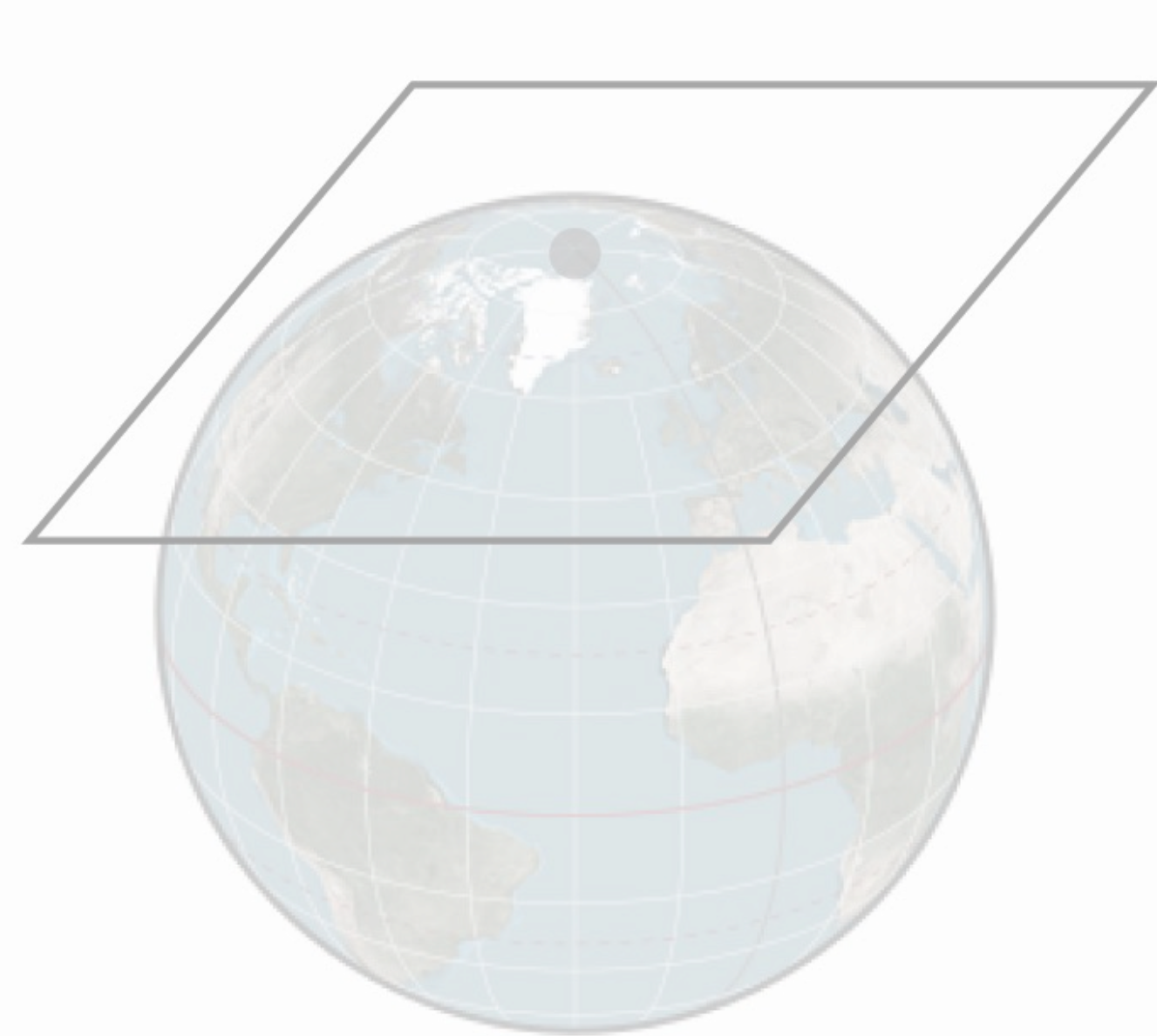
CSV

Shapefiles - old classic

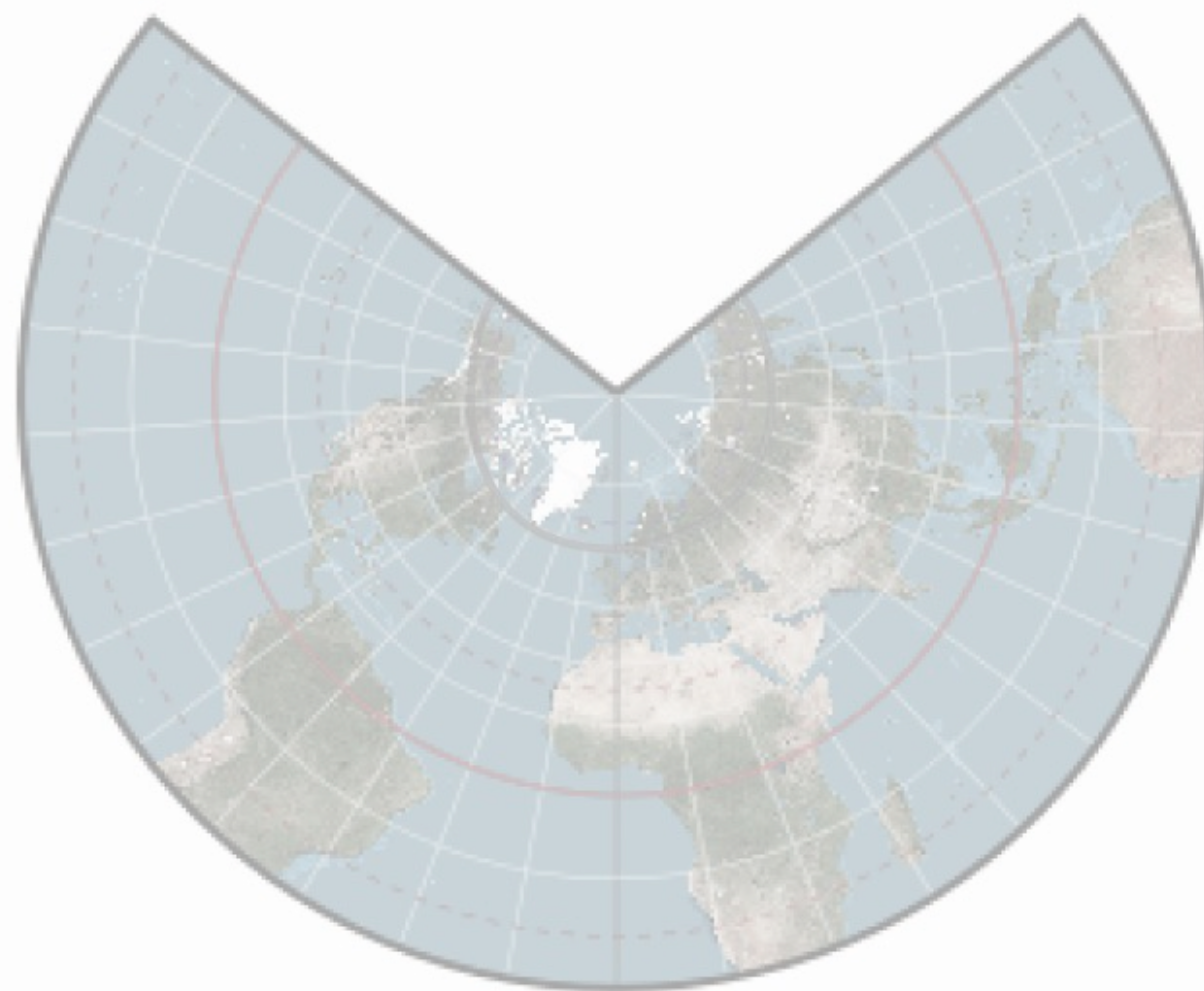
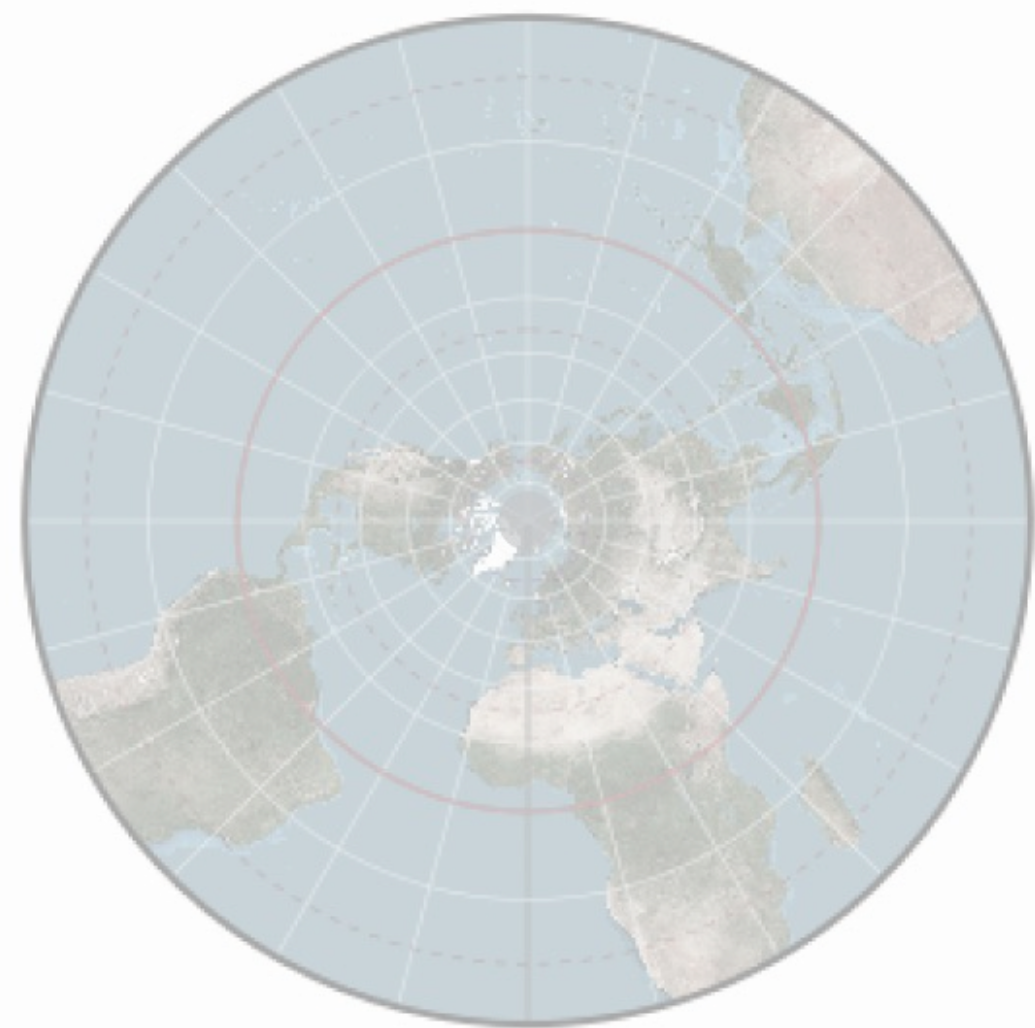
(Geo)Parquet - up & coming

Most common vector formats used:





Coordinate Reference Systems

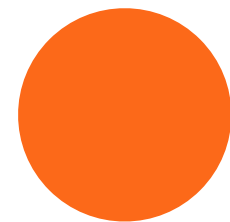


Coordinate Reference System (CRS)

"Without a CRS, the geometries would simply be a collection of coordinates in an arbitrary space"

Coordinate Reference System (CRS)

Point(x, y)



???



Geographic reference systems (GRS)

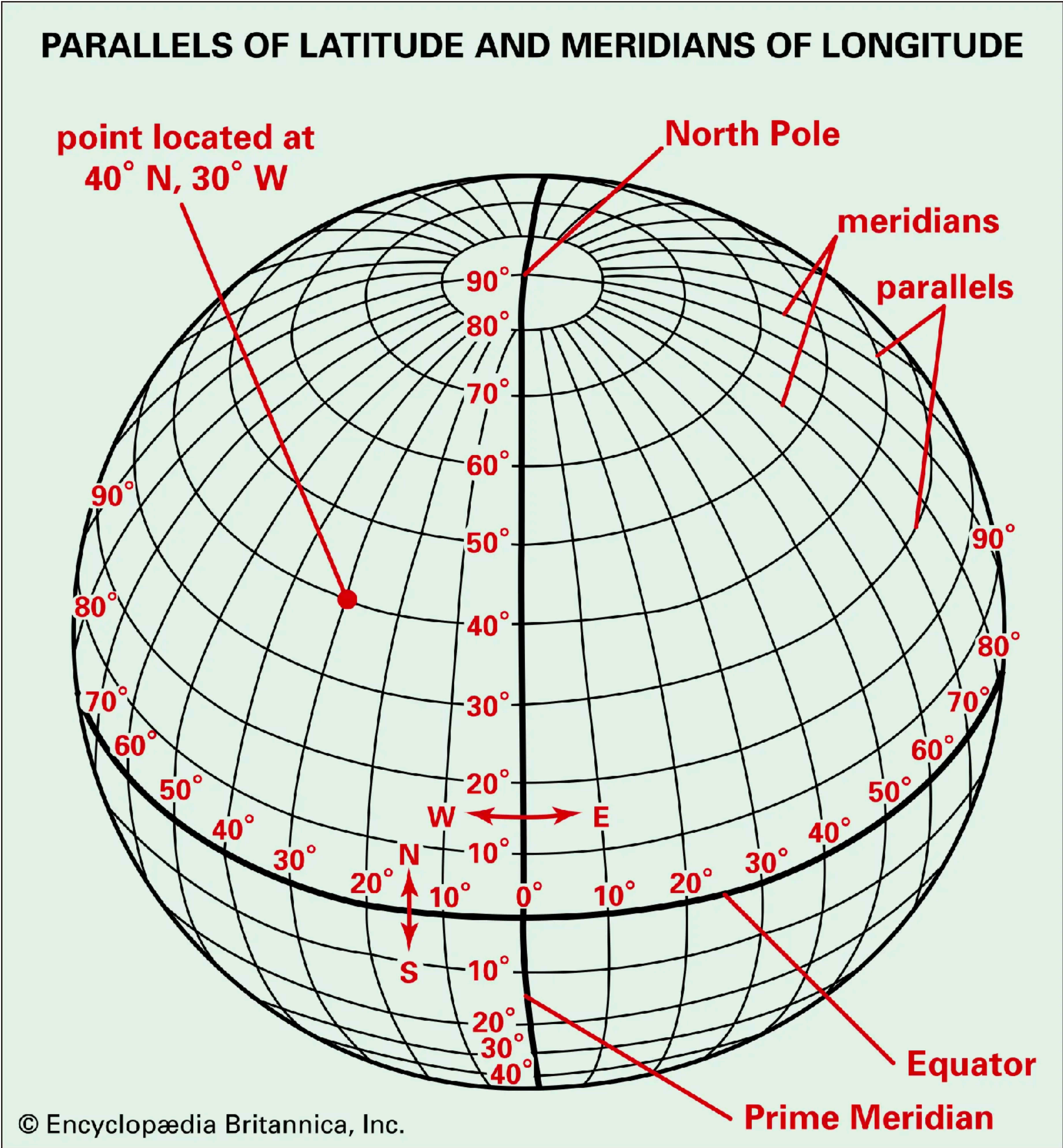
Longitude x is constant on meridians

Latitude y is constant on parallels

Units: Degrees

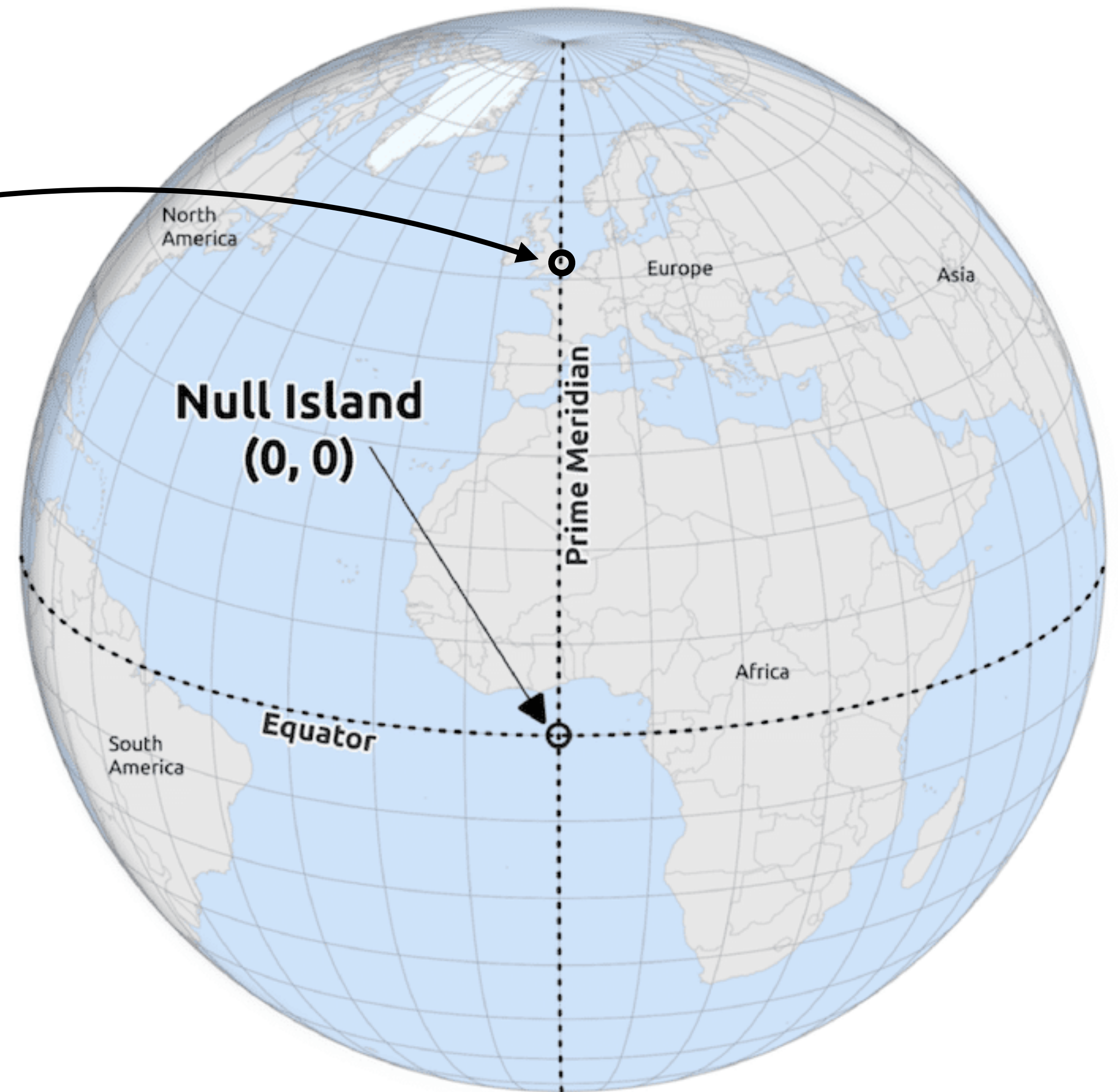


12.591074 E 55.659632 N
x y



The **Prime Meridian** and the **Equator** define the origin 0,0

The Greenwich observatory defines the prime meridian



0 E 51.476852 N

Geographic RS are well defined worldwide..

Geographic RS



$$(\phi, \lambda) = (-0.1, 51.5) = 0.1 \text{ W } 51.5 \text{ N}$$

Longitude	Degrees W/E from meridian
Latitude	Degrees N/S from equator

..but distance calculations with degrees are not ideal

Geographic RS



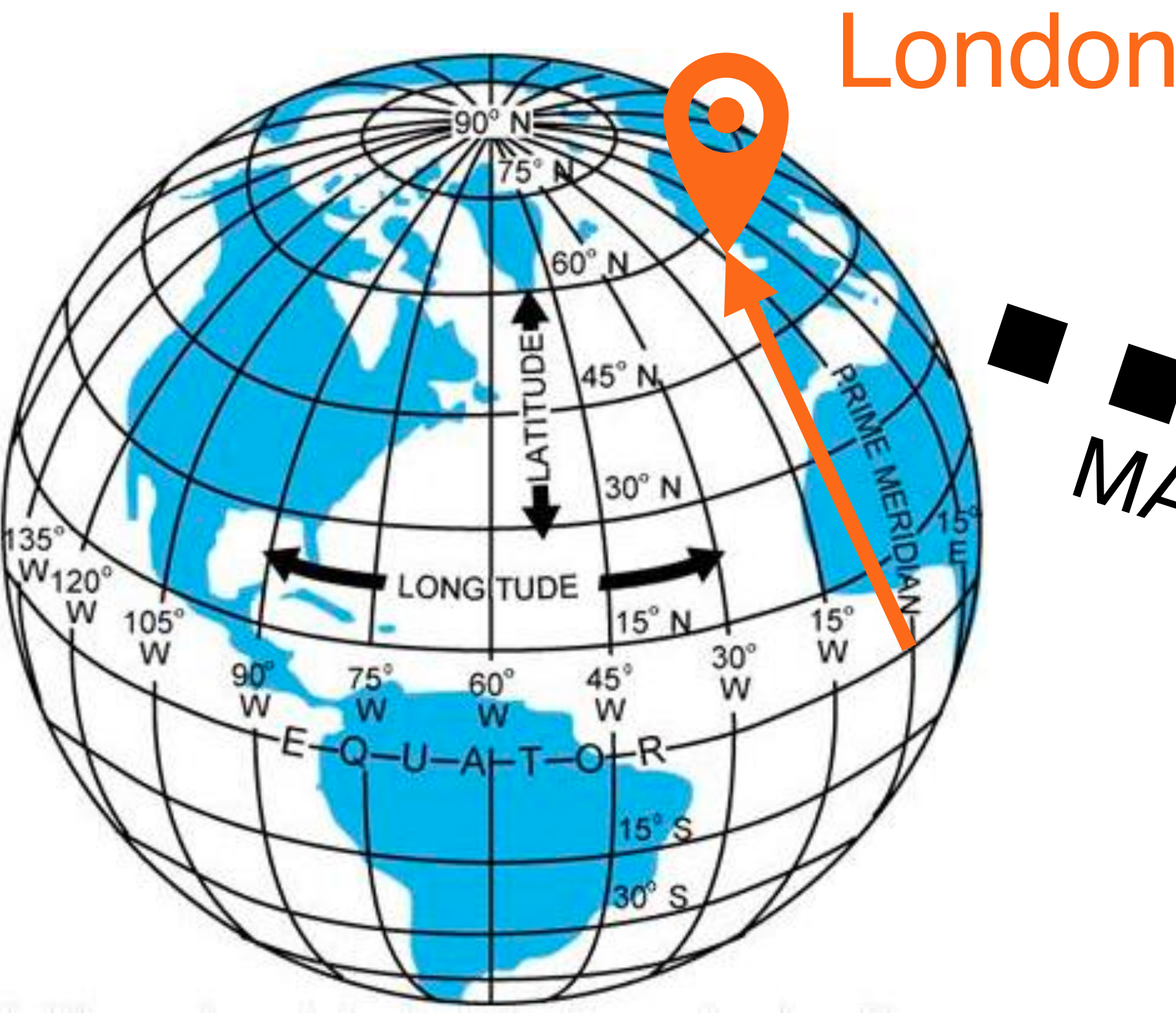
"ITU is 0.03 degrees away from Tivoli"

$$(\phi, \lambda) = (-0.1, 51.5)$$

Longitude	Degrees W/E from meridian
Latitude	Degrees N/S from equator

We use Projected RS to get flat distance in meters

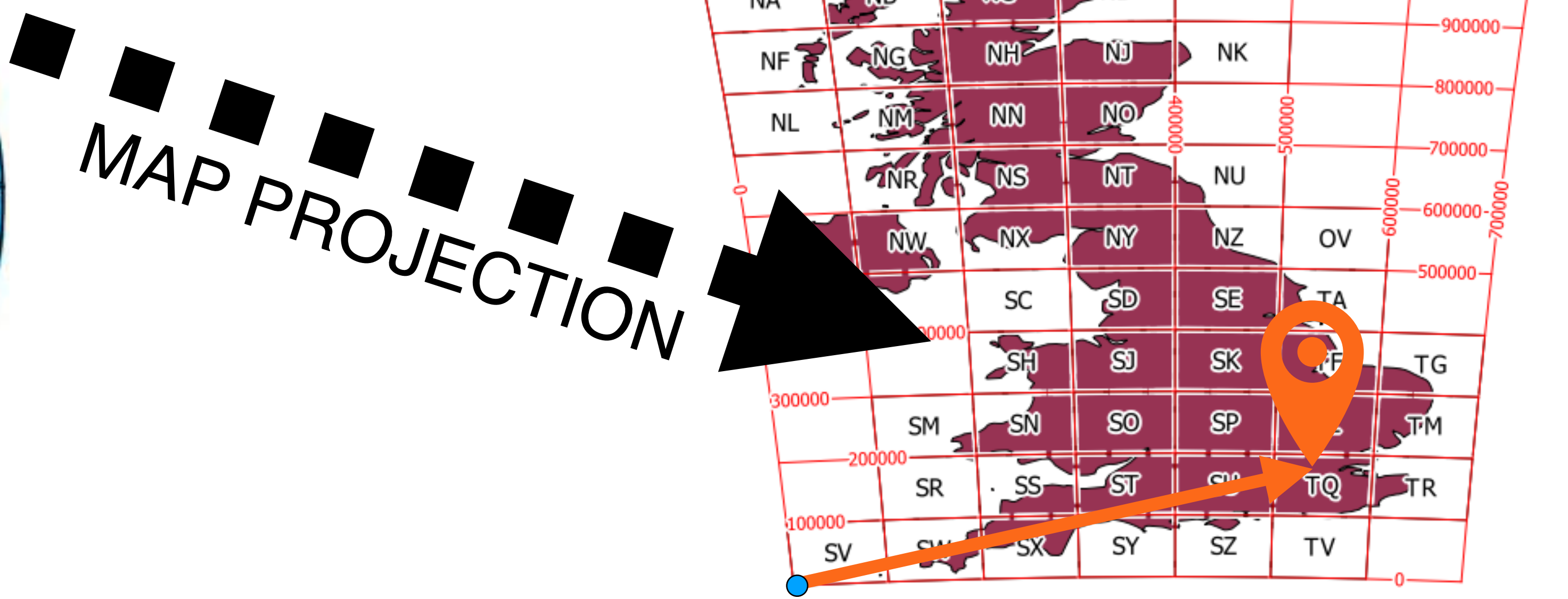
Geographic RS



$$(\phi, \lambda) = (-0.1, 51.5)$$

Longitude Degrees W/E from meridian
Latitude Degrees N/S from equator

Projected RS



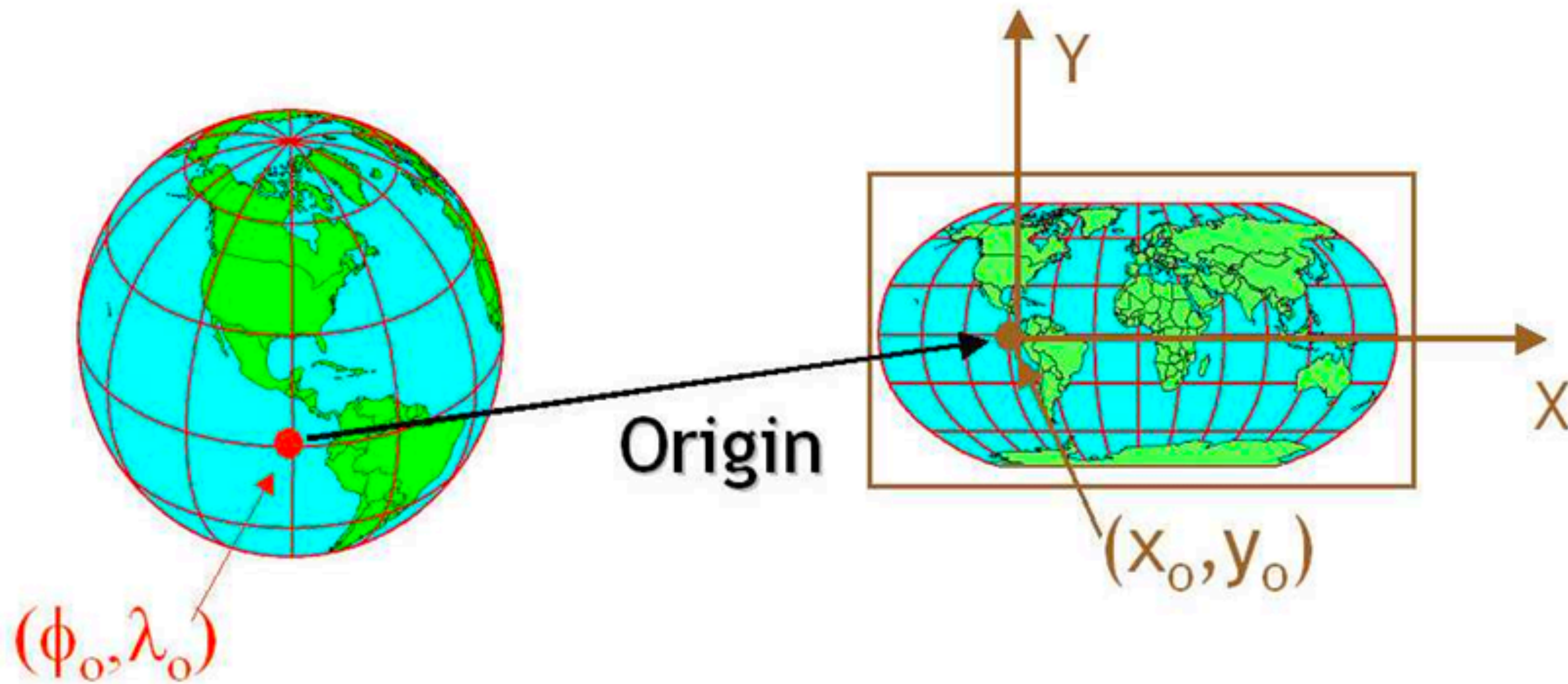
$$(x,y) = (530000, 180000)$$

Easting Meters east from origin (bottom left)
Northing Meters north from origin (bottom left)

Projected reference systems draw a sphere on a surface

Map projection: “the process of transforming angular (spherical / elliptical) coordinates into planar coordinates.”

Battersby, S. (2017)

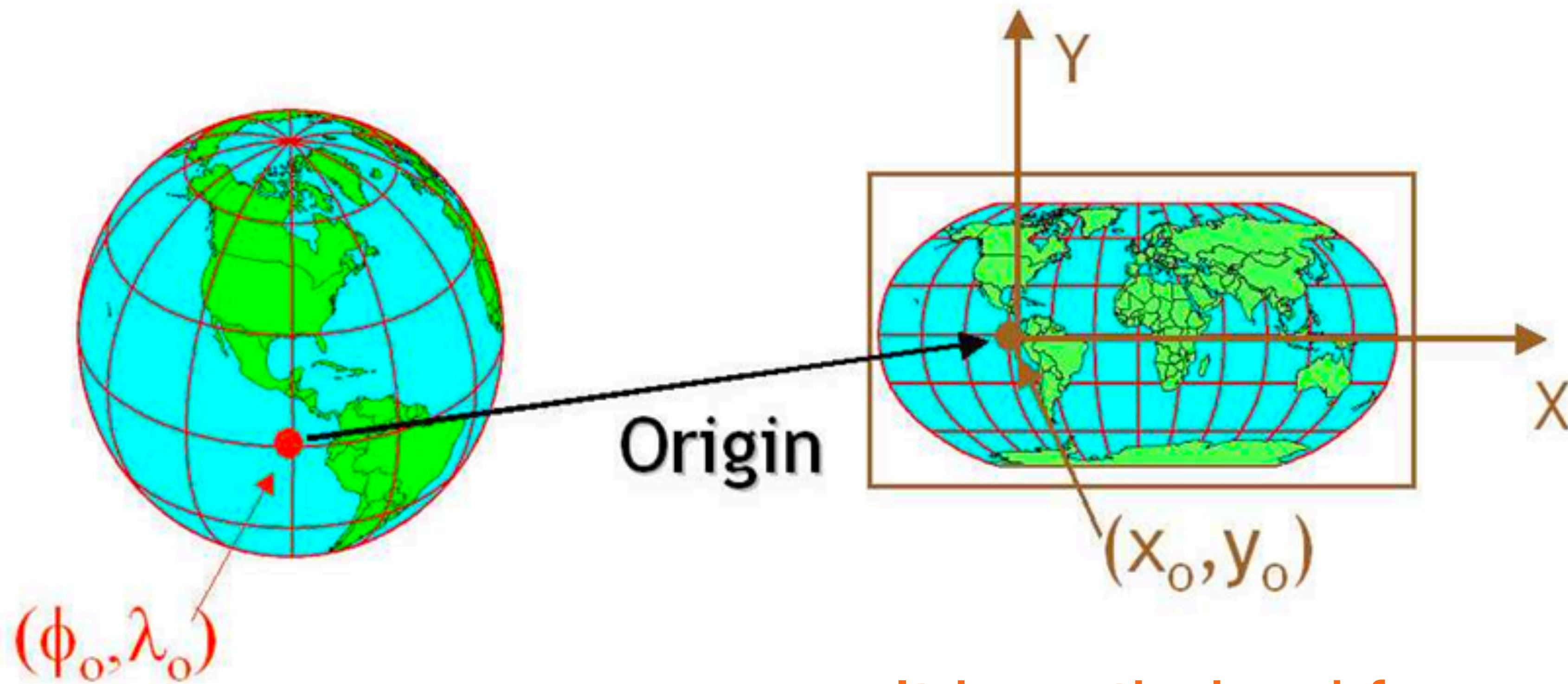


Units usually in **meters**

Projected reference systems draw a sphere on a surface

Map projection: “the process of transforming angular (spherical / elliptical) coordinates into planar coordinates.”

Battersby, S. (2017)

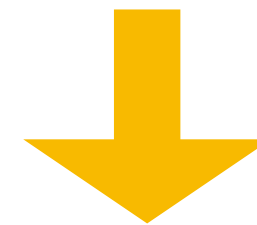


It is optimized for
a specific location

Due to tearing and stretching, all map projections are wrong



"All models are wrong, but some are useful" (George E P Box)



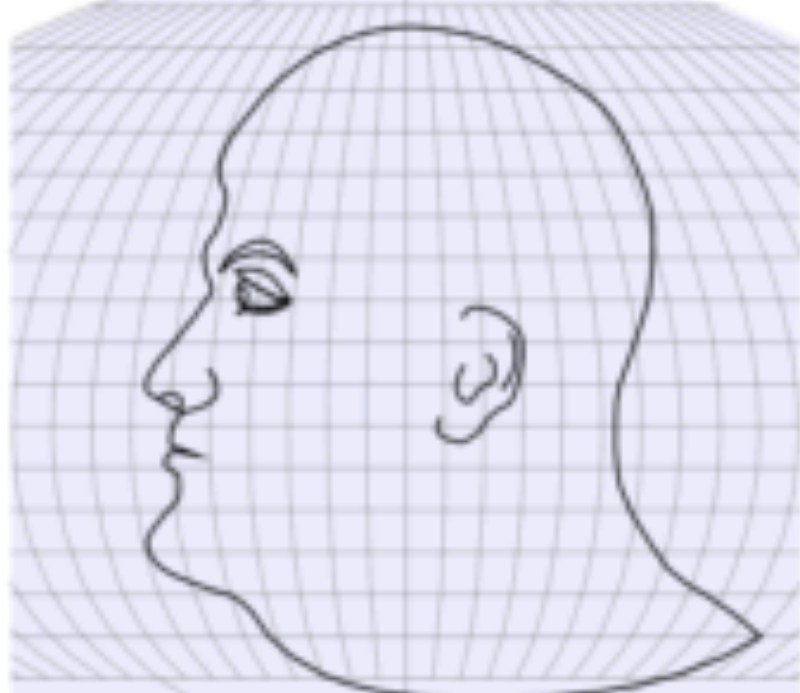
"All maps are wrong, but some are useful" (Keith Ord)

All map projections are wrong

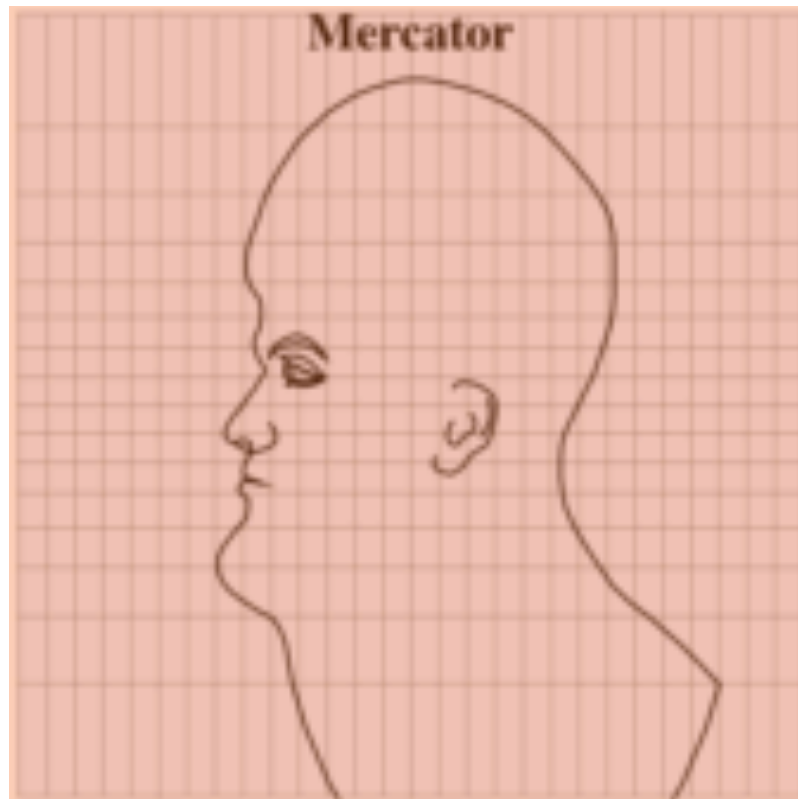
Laskowski



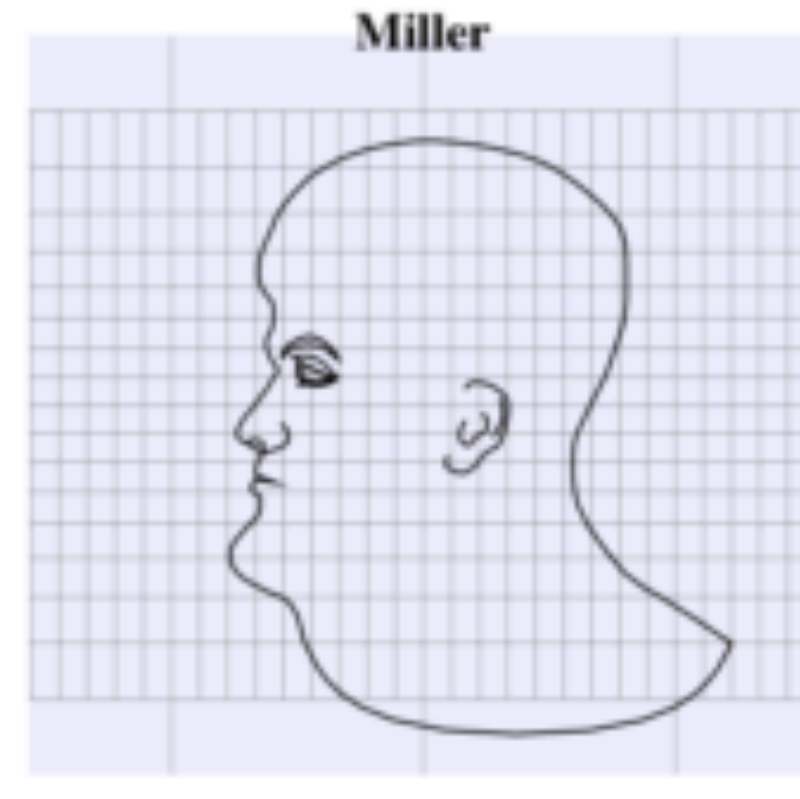
Loximuthal



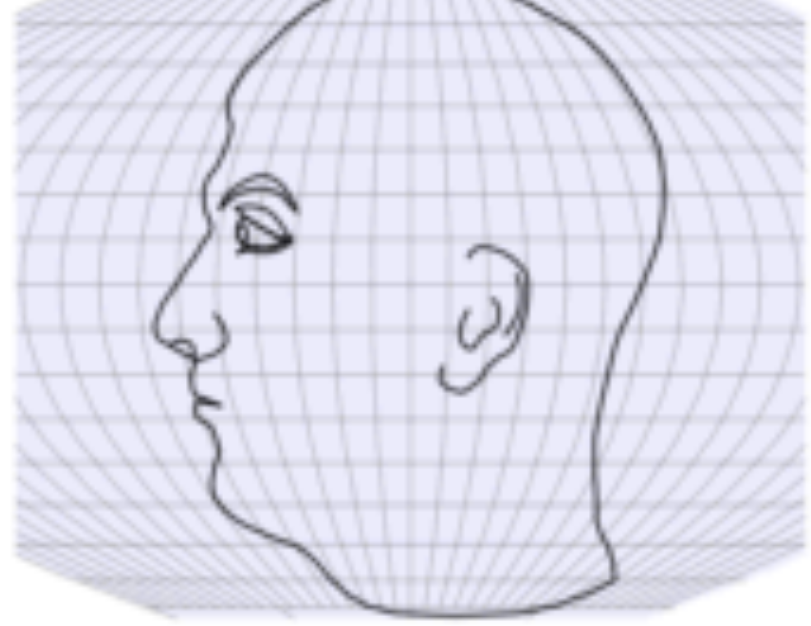
Mercator



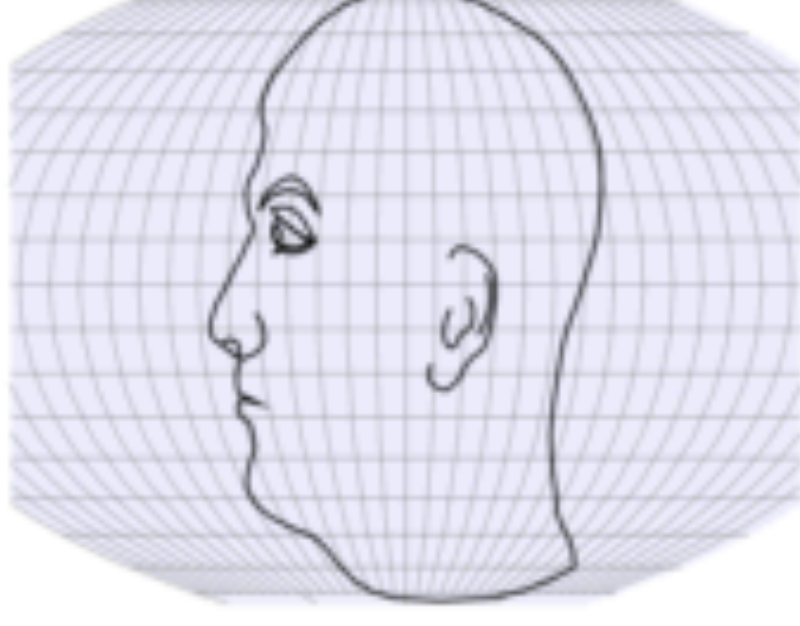
Miller



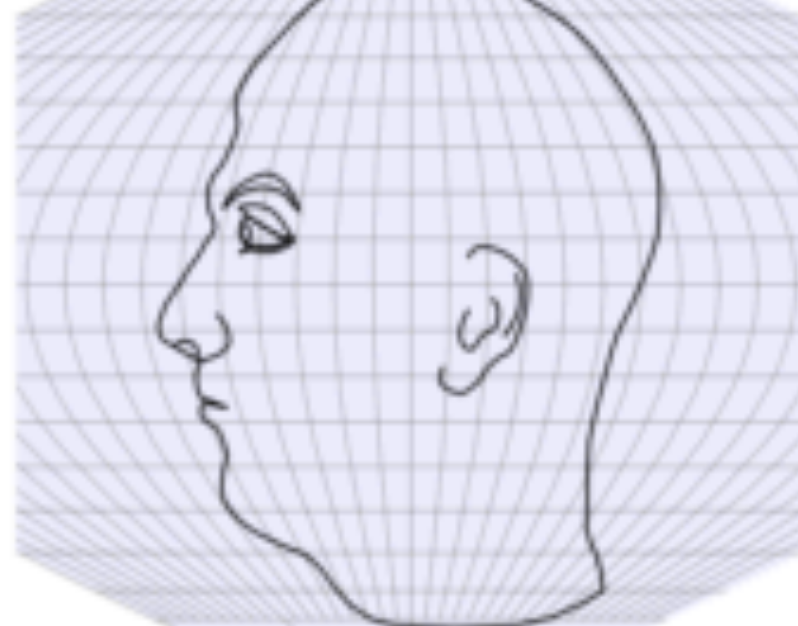
McBryde-Thomas Flat-Polar
Parabolic



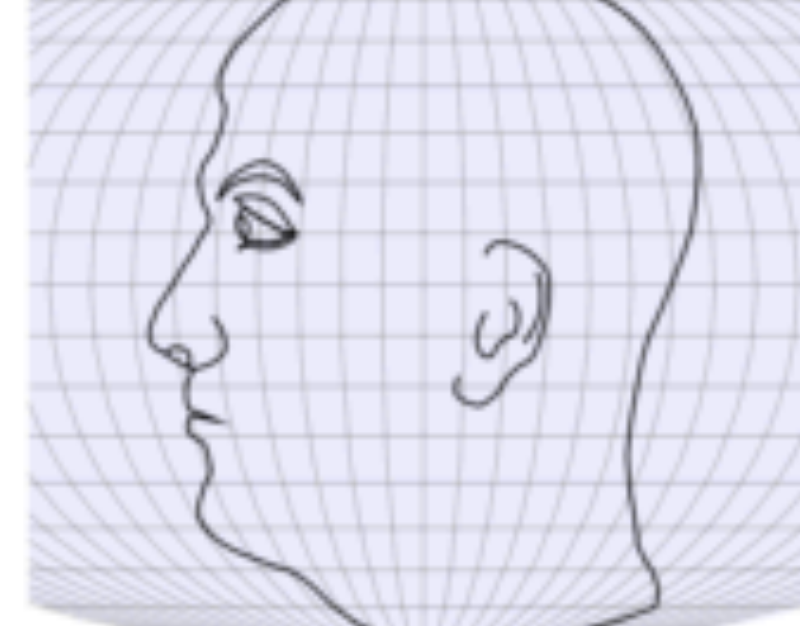
McBryde-Thomas Flat-Polar
Quartic



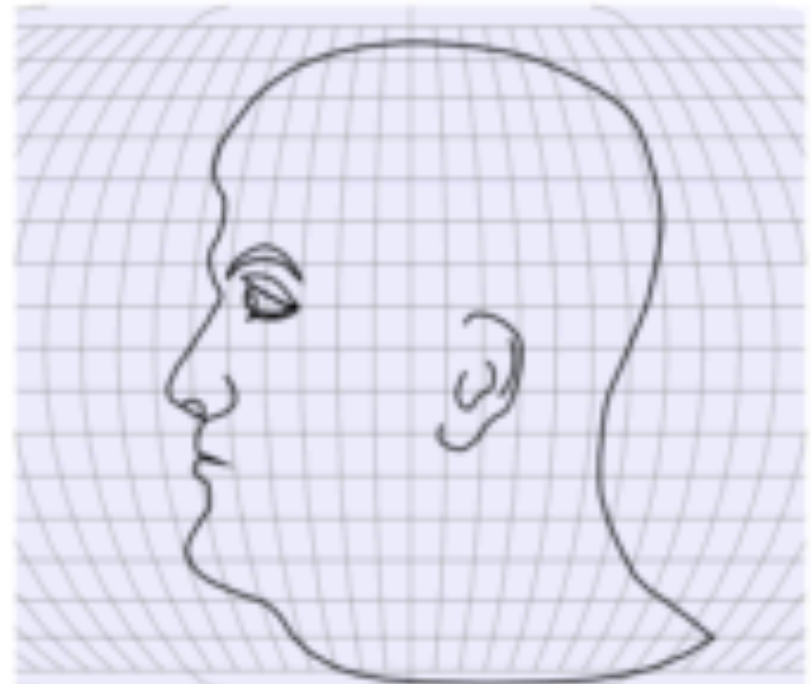
McBryde-Thomas Flat-Polar
Sinusoidal



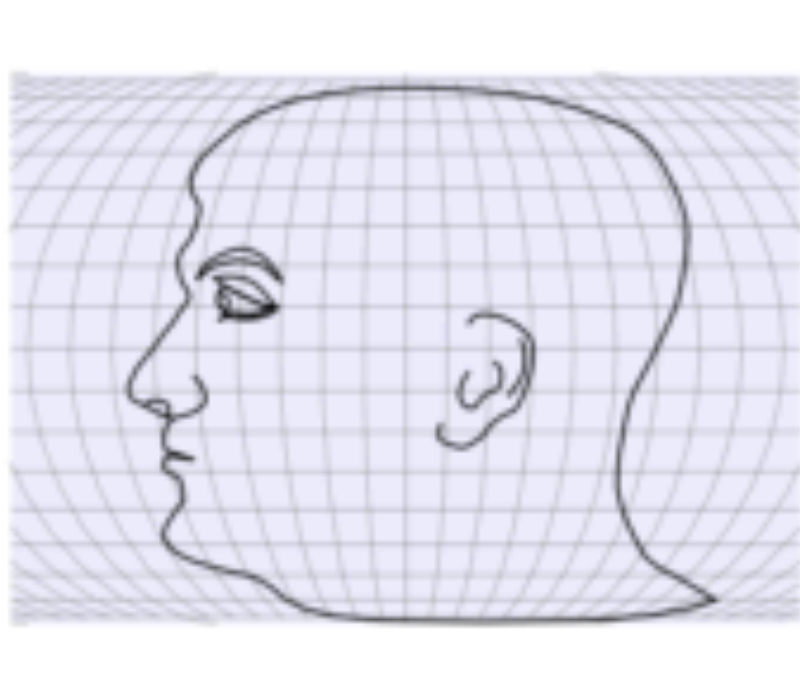
Mollweide



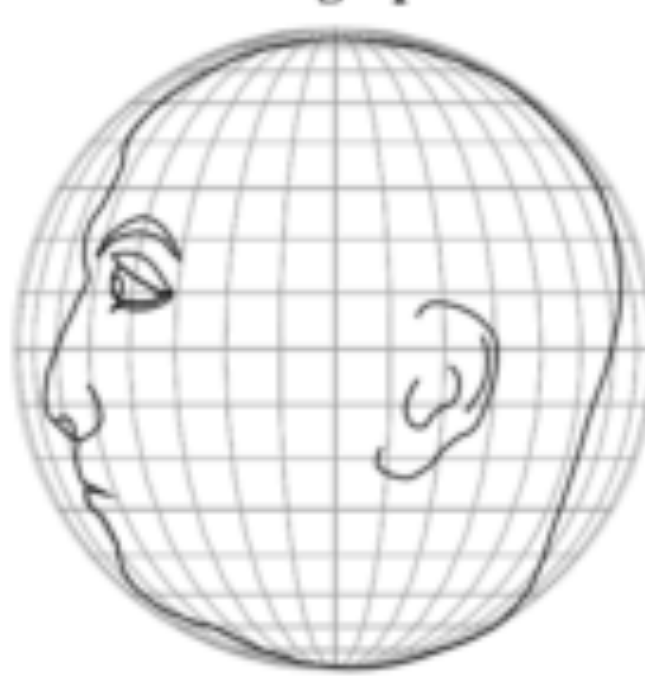
Natural Earth



Nell-Hammer



Orthographic



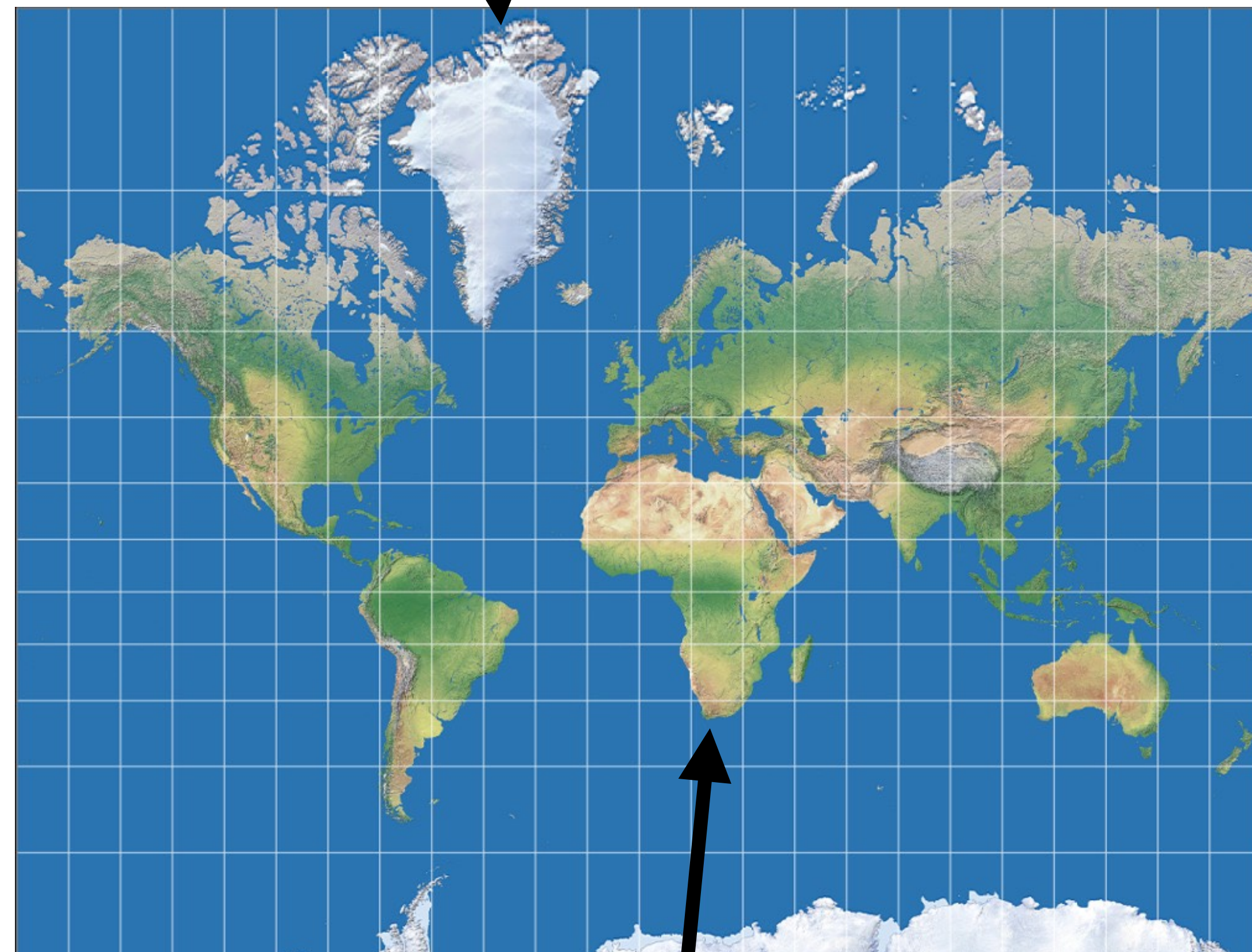
Polyconic



All map projections are wrong. Example: Mercator



Greenland: 2 Mio km²



Afrika: 30 Mio km²

What can matter when designing a map?

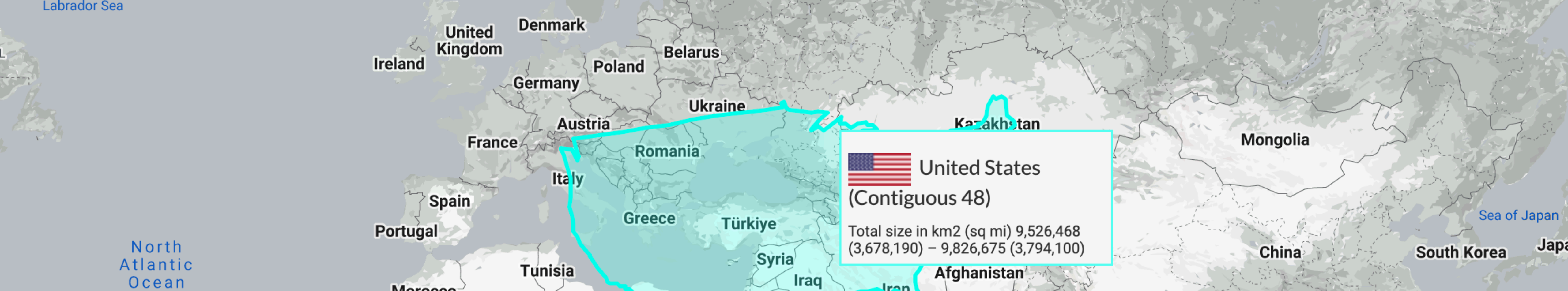
What can matter when designing a map?

- Looking good ✨
- Be easy to read 🧩
- Focusing on one area of interest 🎯
- Be useful to navigate 🧭
- Be useful to compare areas or distances 📐
- ..?

Map projections matter!



<https://youtu.be/vVX-PrBRtTY?si=95vwxOk7nqYkEvAM&t=42>

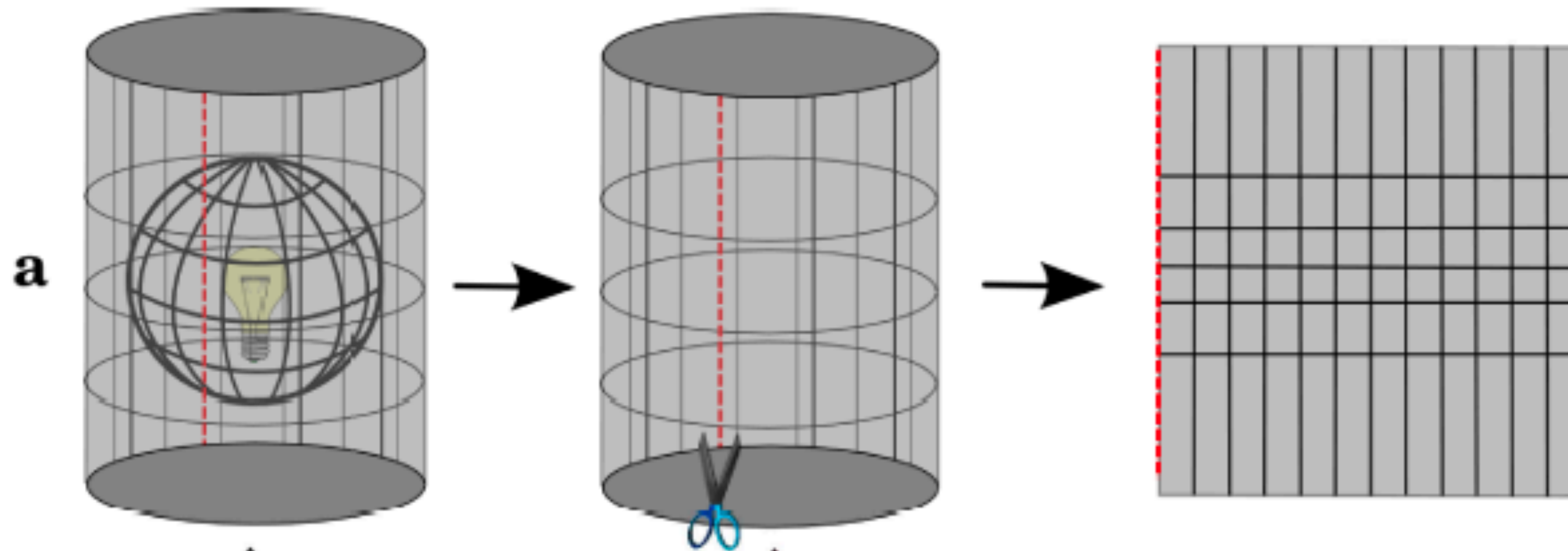


THE TRUE SIZE OF...

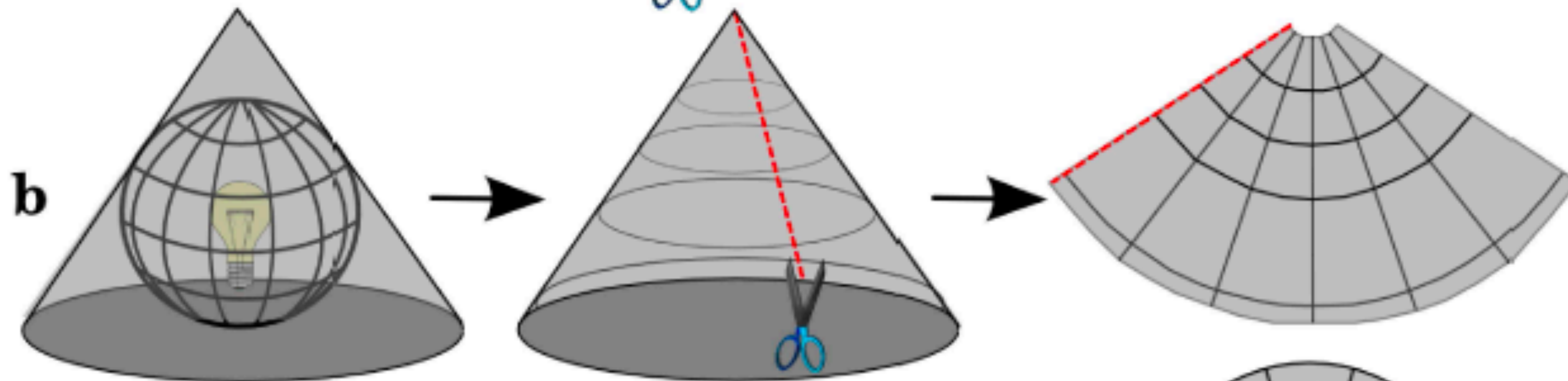


3 families of map projections

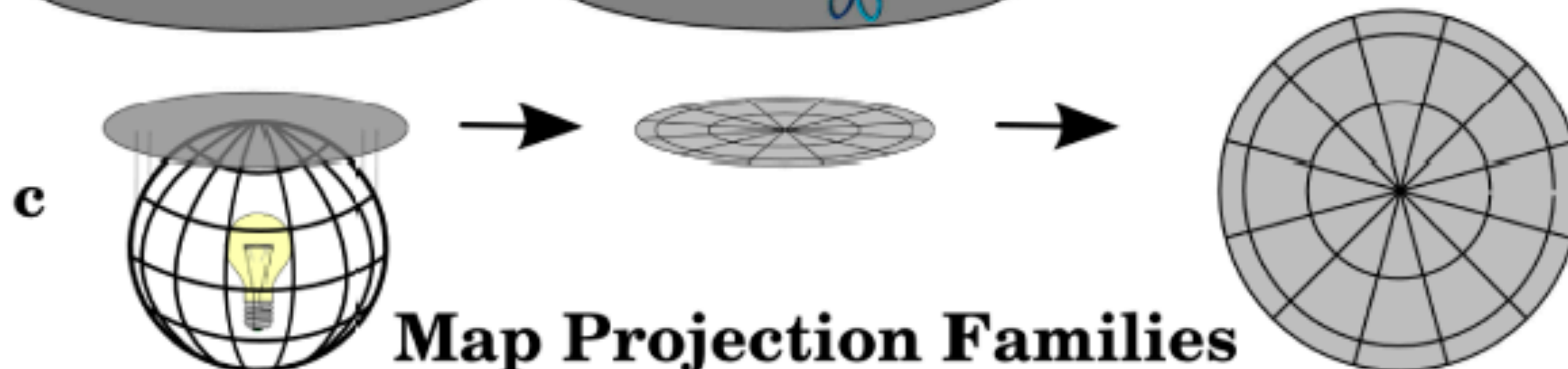
Cylindrical



Conical

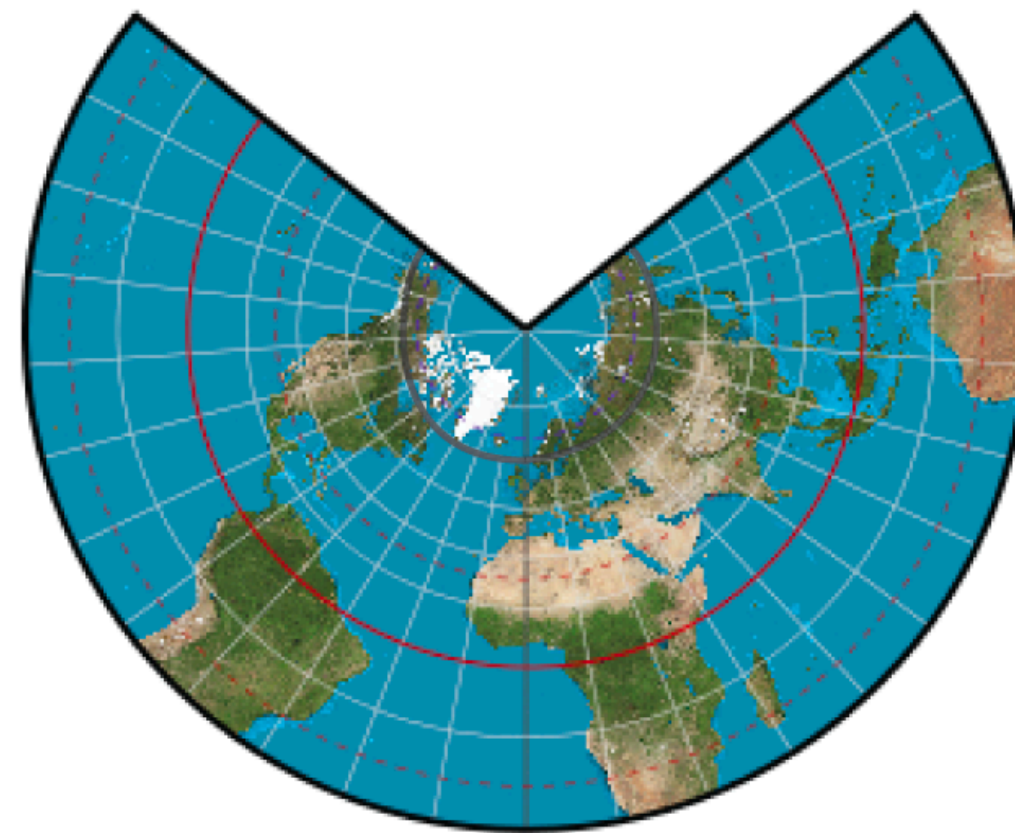
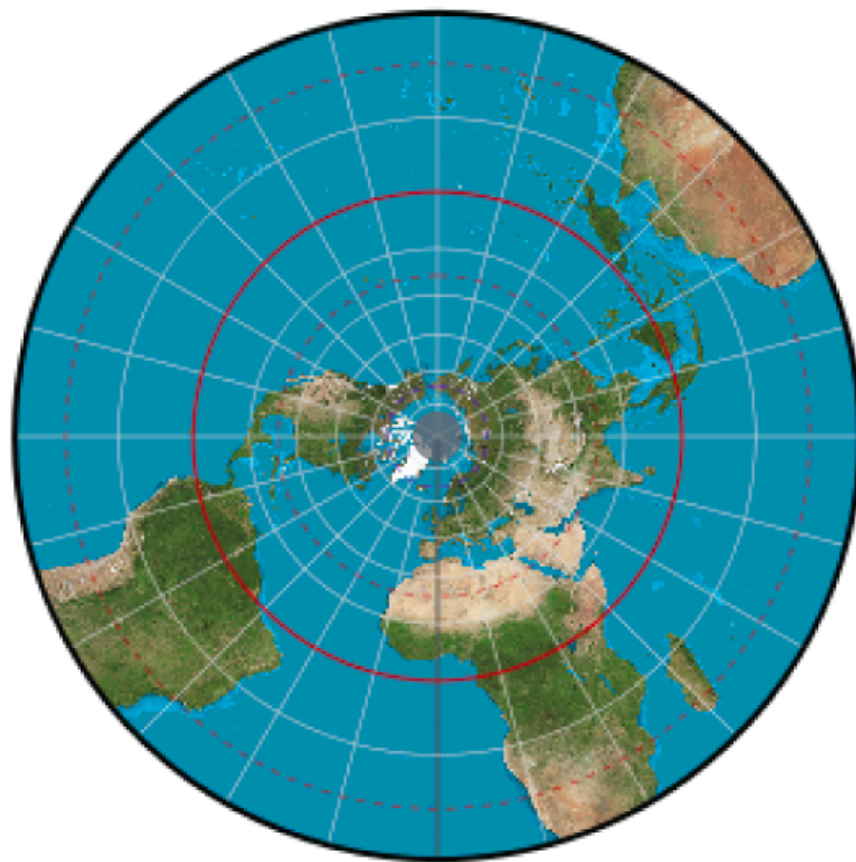
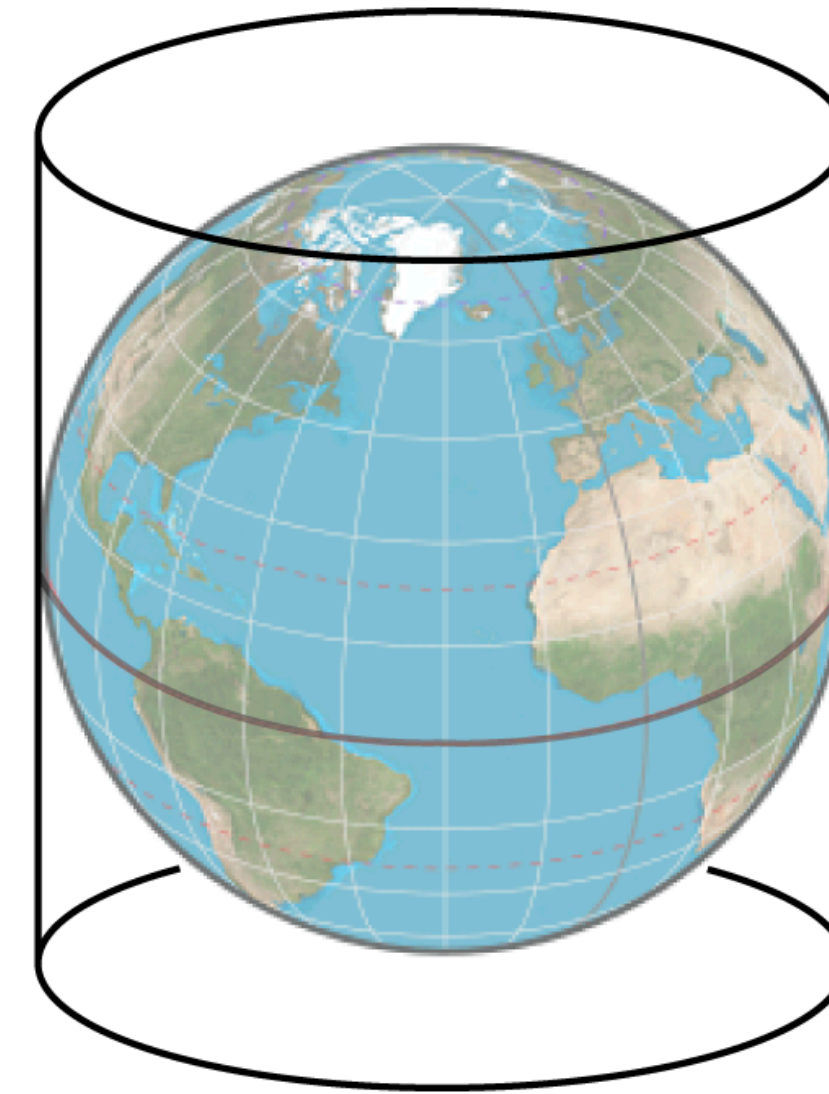
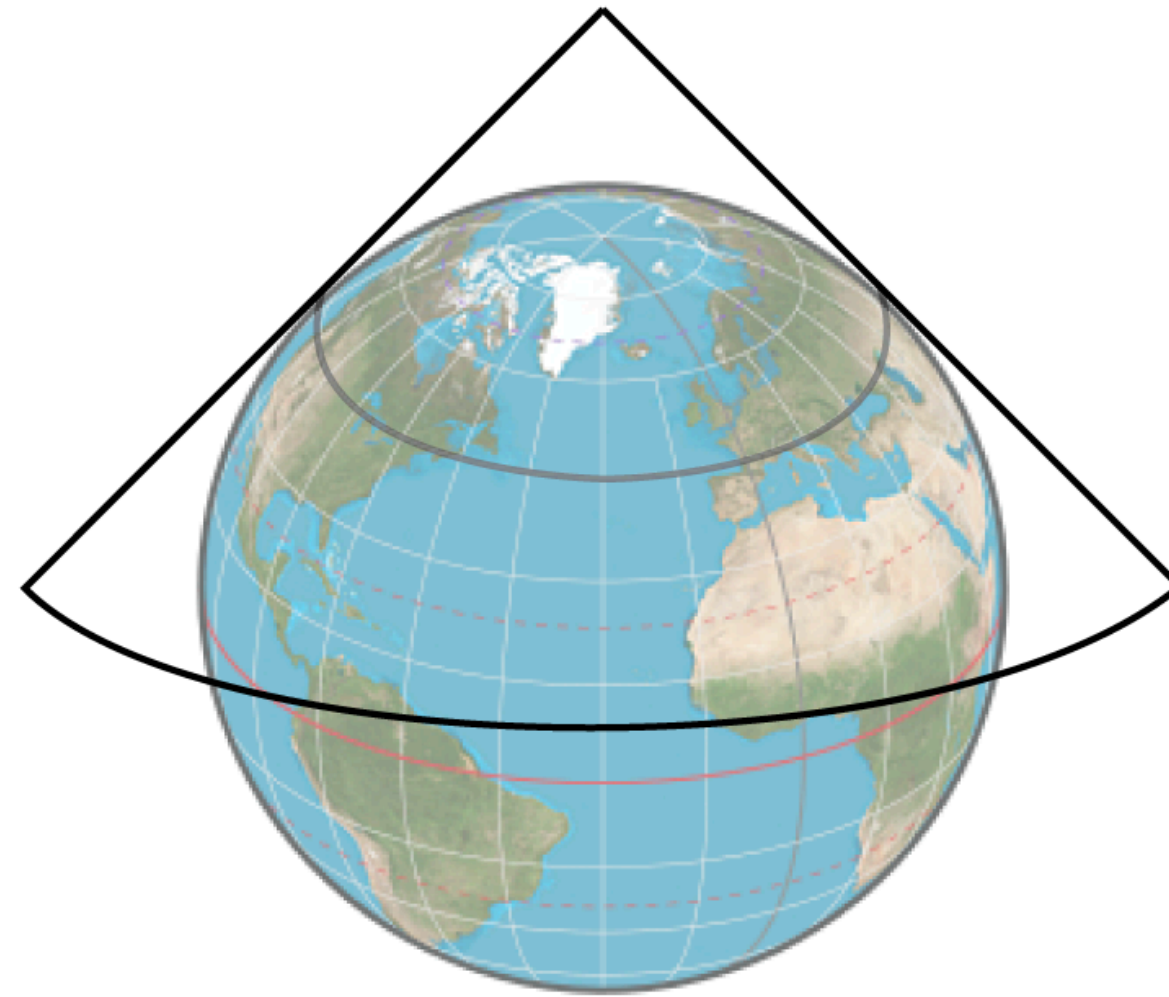
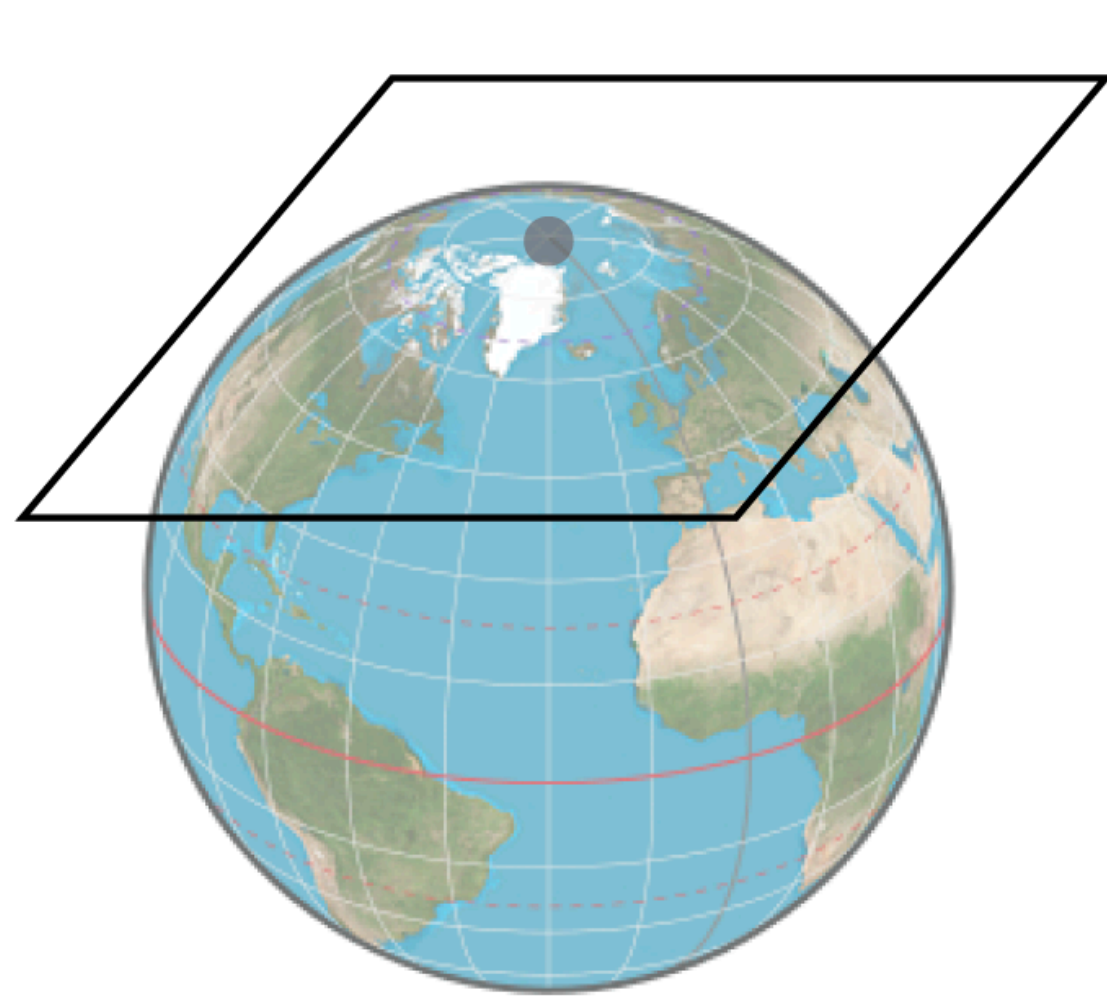


Planar



Map Projection Families

3 families of map projections



Map projections are optimized for different things

Azimuthal: preserve directions from the center to any other point on the plane.

Conformal: preserve local angular relationships across the projection.

Equal area or equivalent: preserve relative areas across the projection.

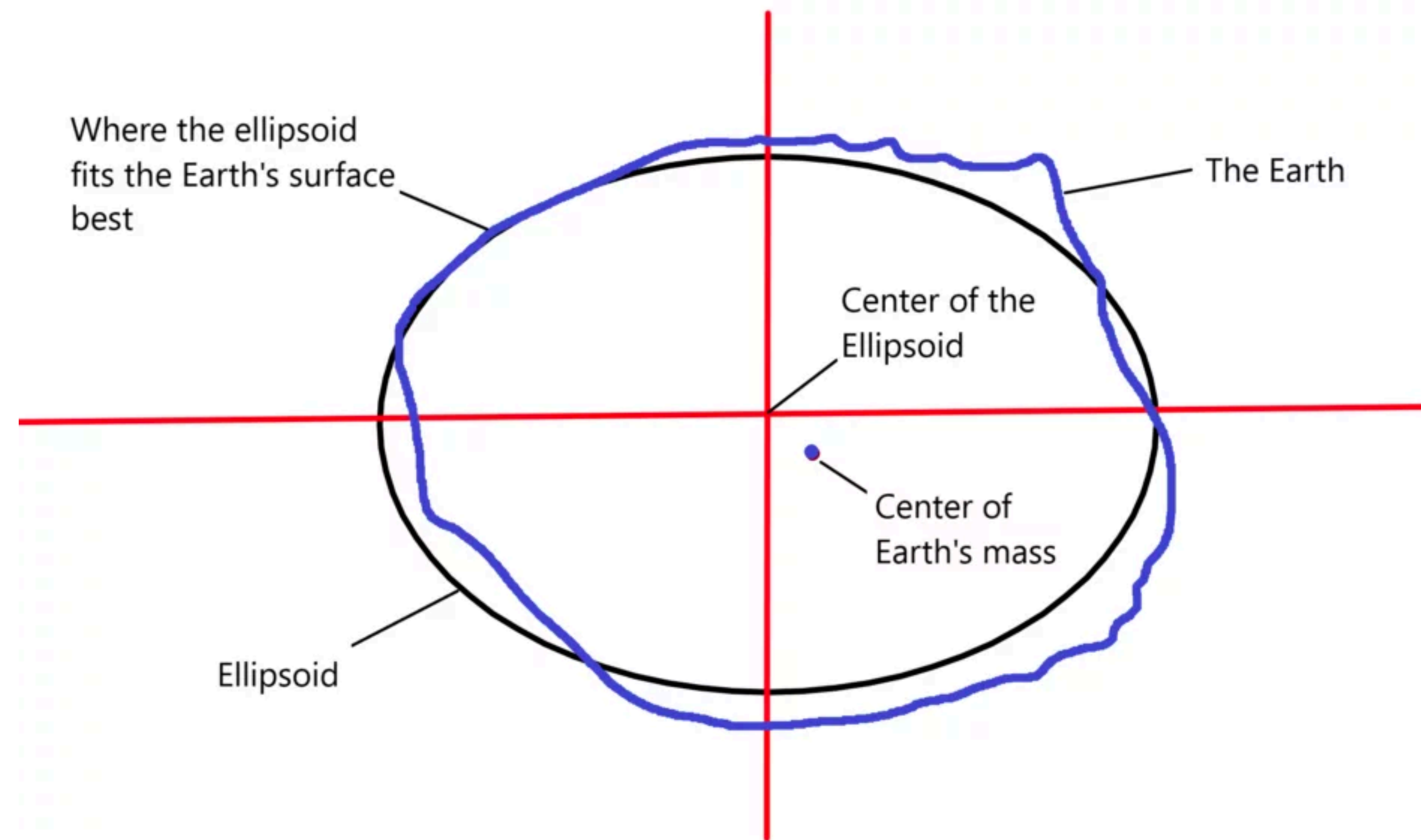
Equidistant: preserve distances from one or two specified points to any other point on the plane

Compromise: does not preserve either areas or angles, but finds a compromise between the two, typically to present a more aesthetically pleasing map while reducing excessive distortion

On top of the sphere-to-plane issue, the earth is not round

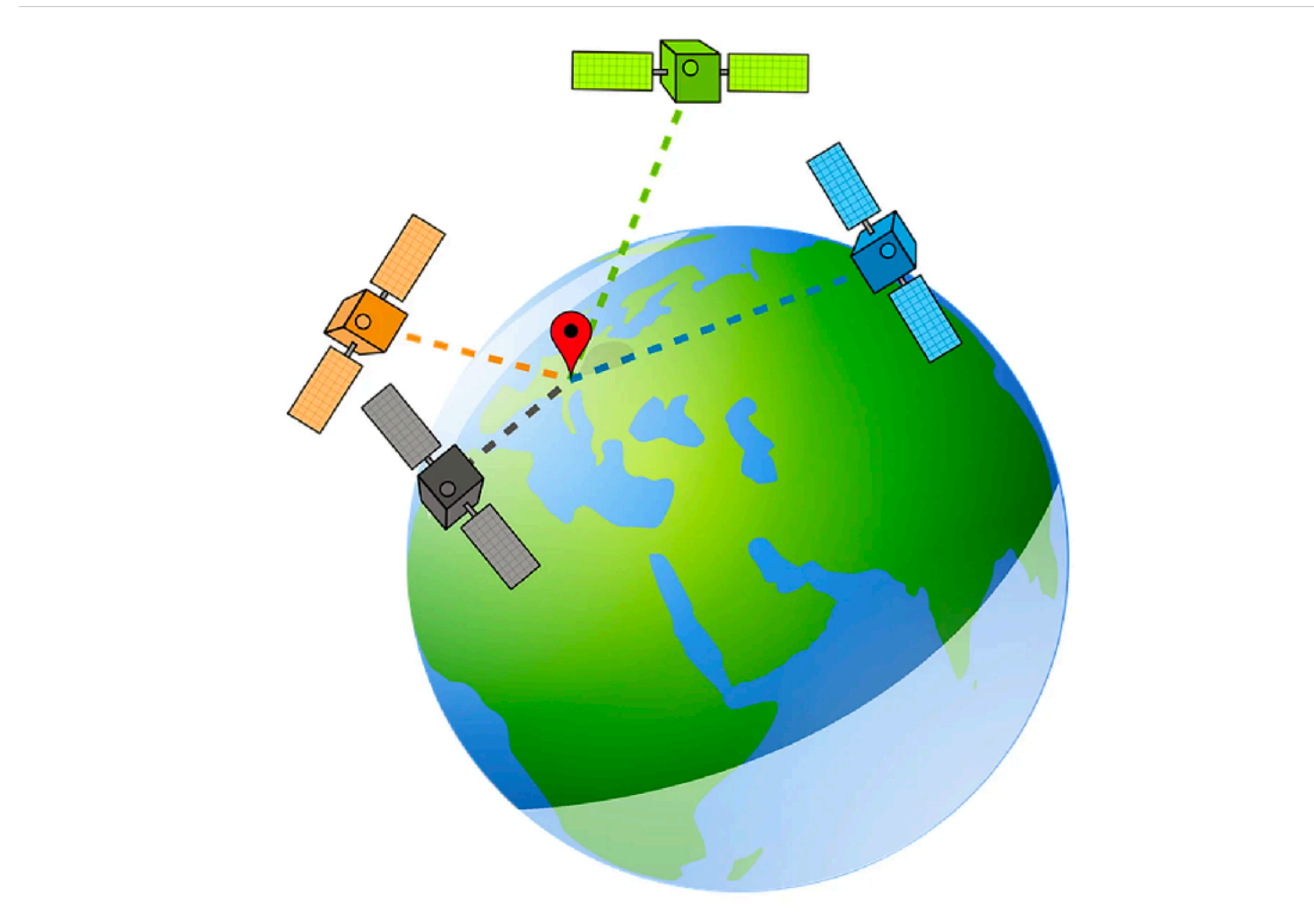
A 'datum' is a model of the shape of the Earth

WGS 84 (World Geodetic System 1984) is the most common one



Most common reference systems are based on **WGS 84**

GRS: WGS 84 (World Geodetic System 1984) / EPSG:4326



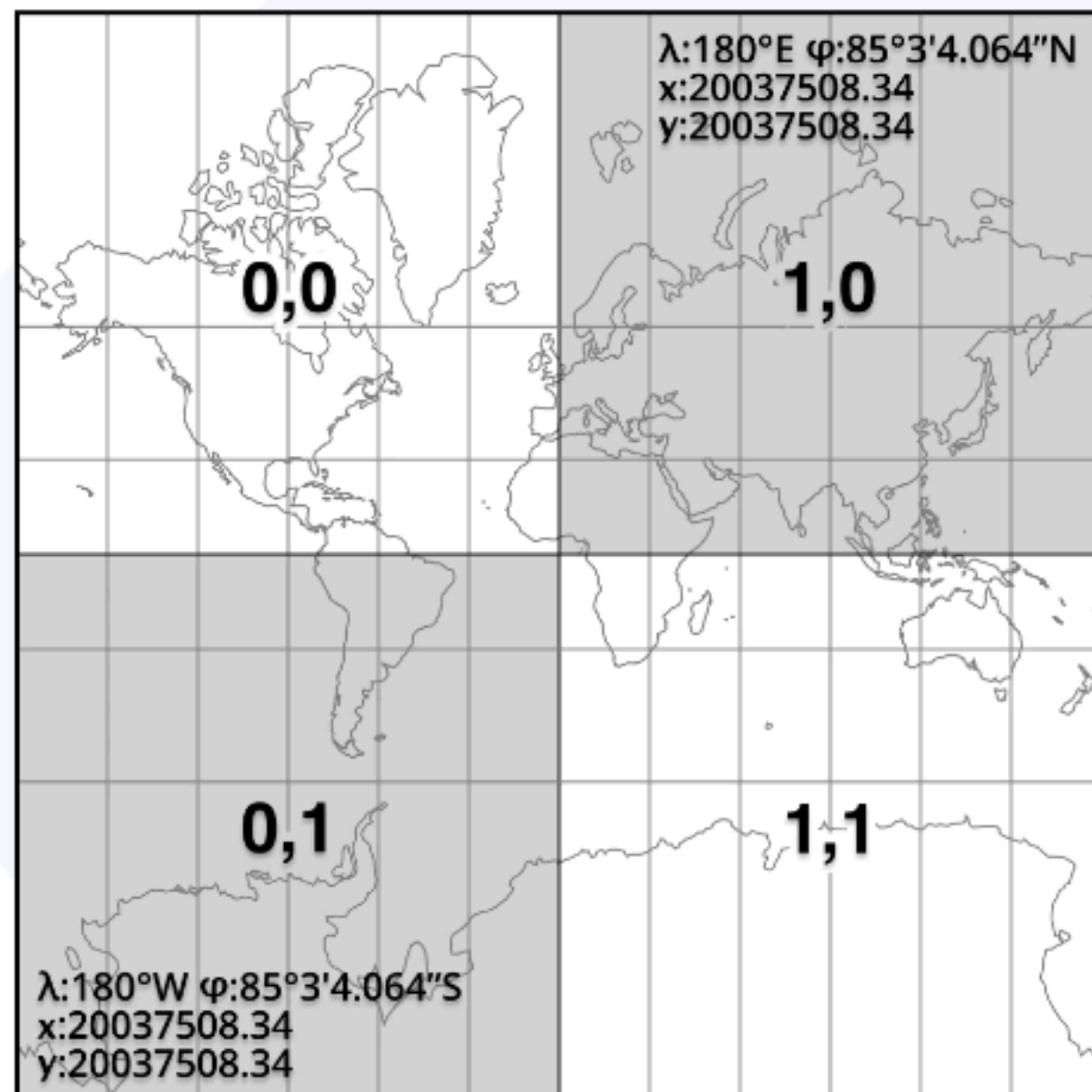
PRS: WGS 84 - Web/Pseudo-Mercator / EPSG:3857



Google Maps

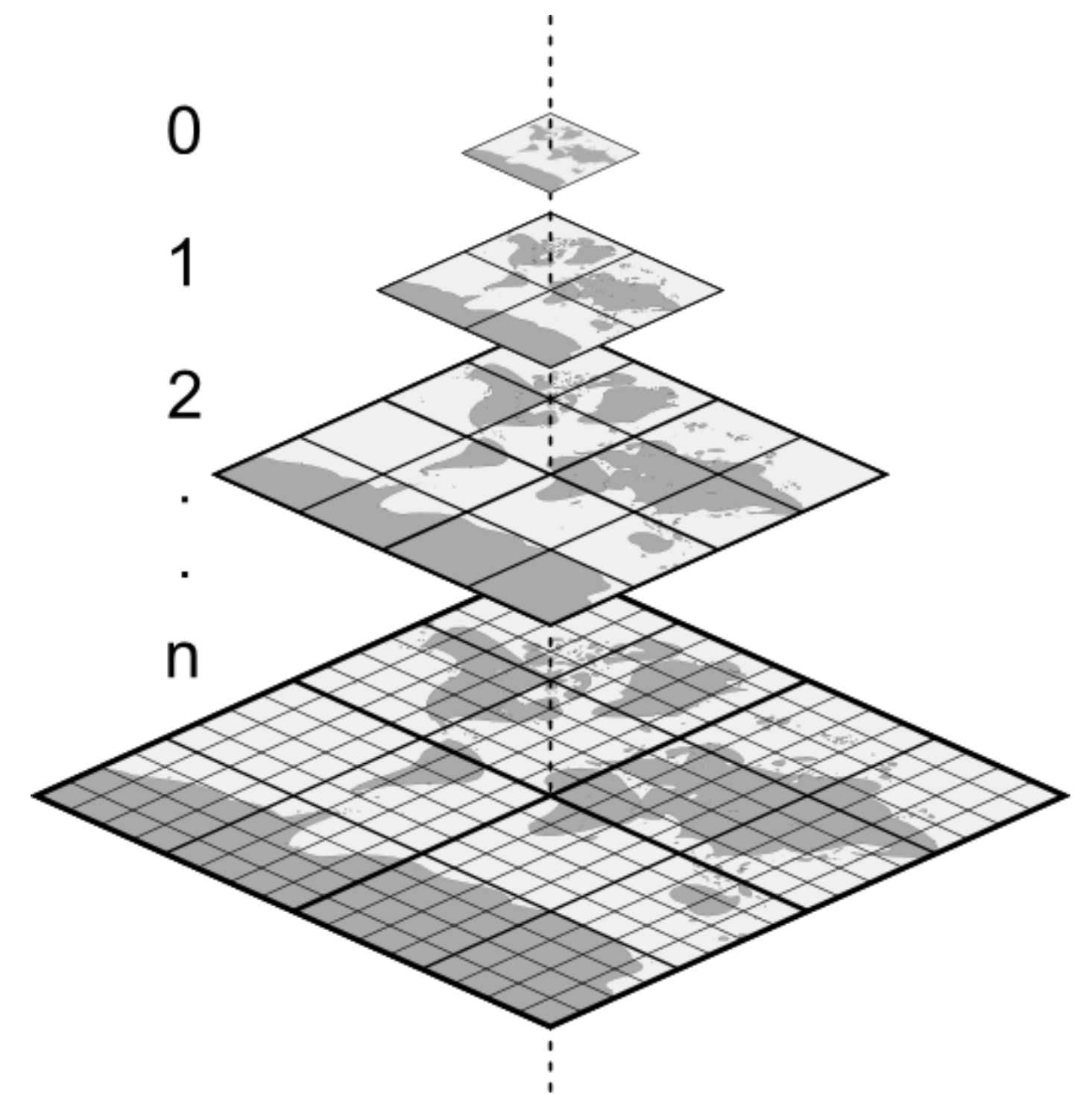
Why is Mercator still around?

Web-Mercator (EPSG:3857) still used for Google Maps, OpenStreetMap, + most other web maps



Map tiles and a spherical datum allow for fast computations

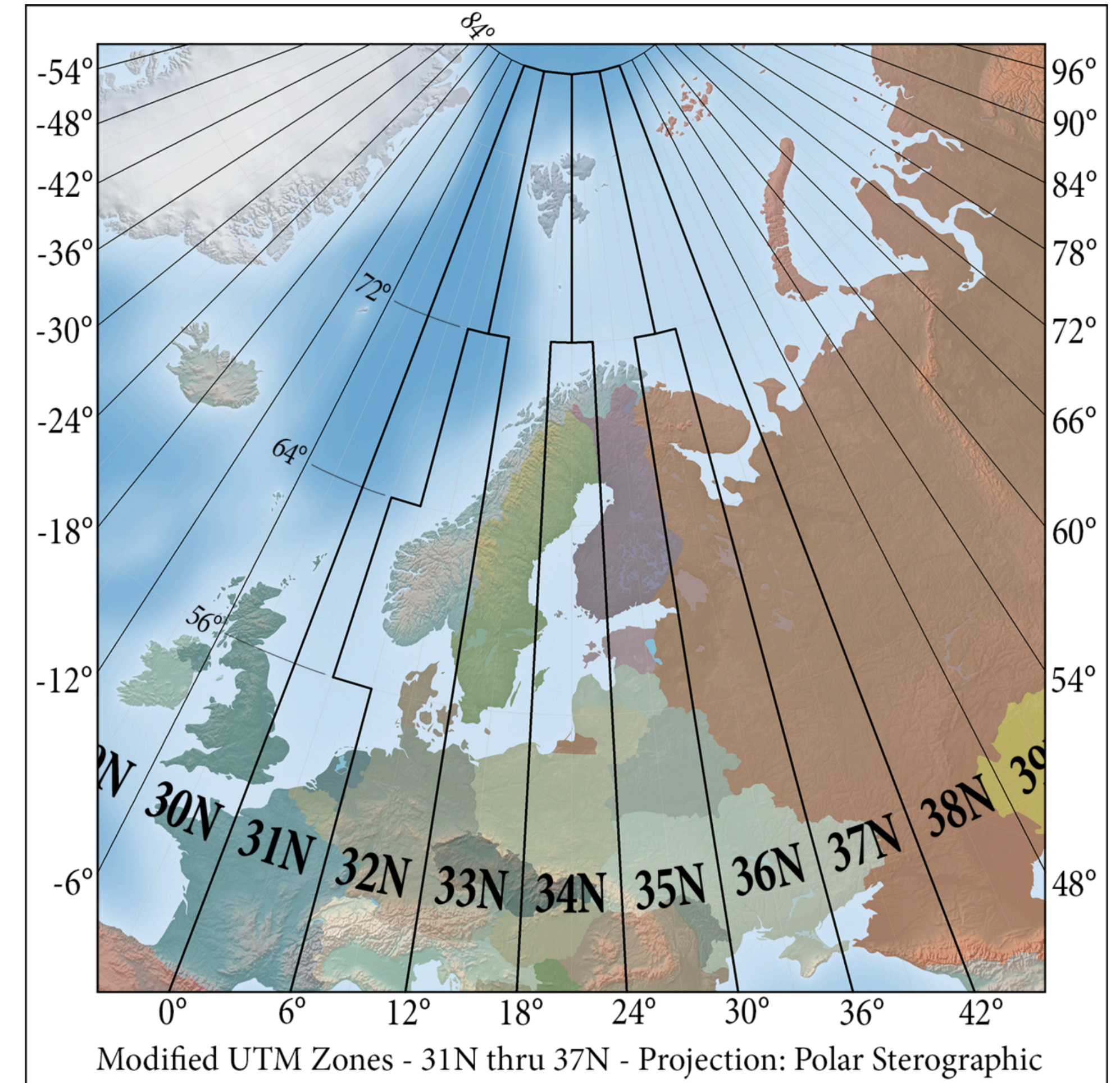
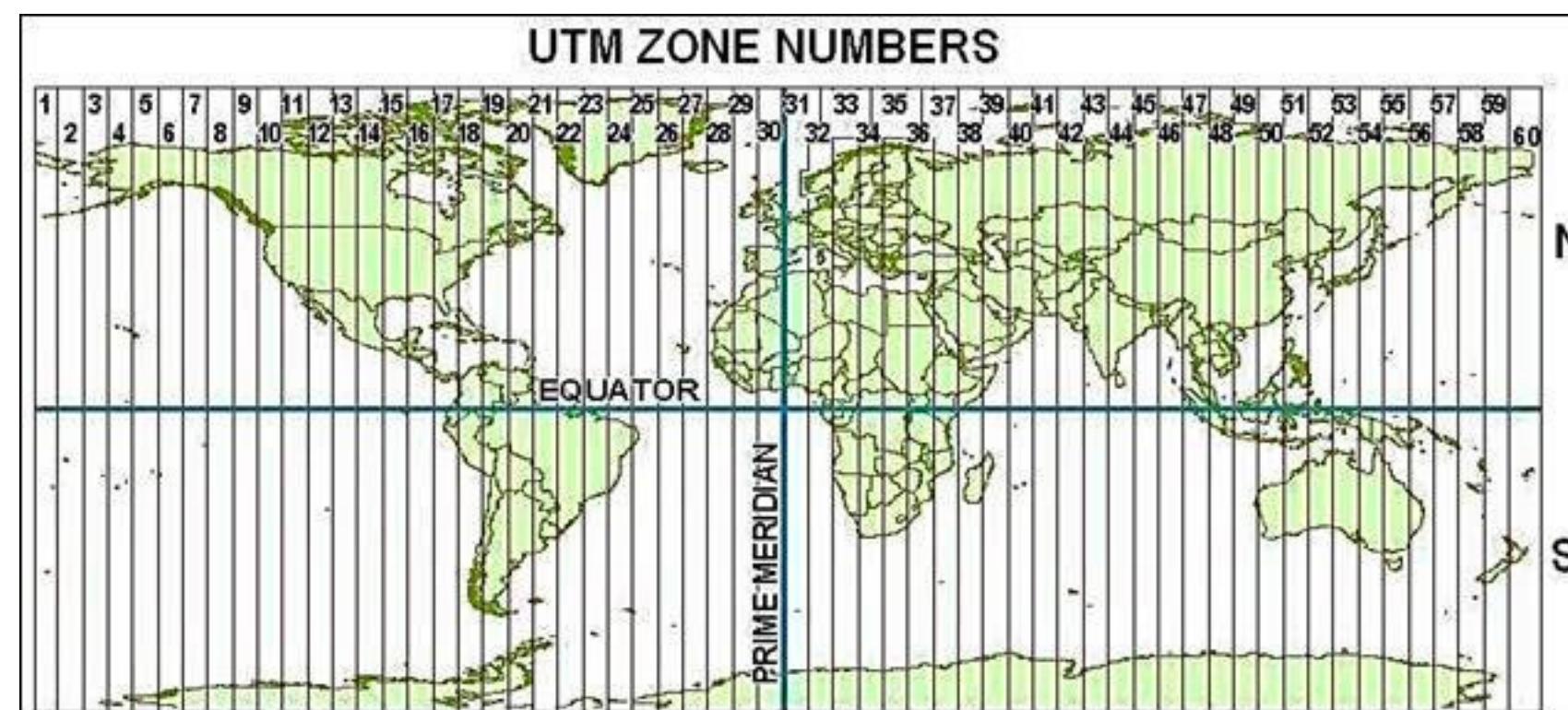
Useful but inaccurate



How to choose your map projection?

- The *location* of your data
- The *purpose* of the map/analysis
- The *norm/tradition*

Web maps: Usually Web-Mercator (EPSG:3857)
EU: Lambert Azimuthal Equal Area (EPSG:3035)
Denmark: UTM Zone32N (EPSG:25832)



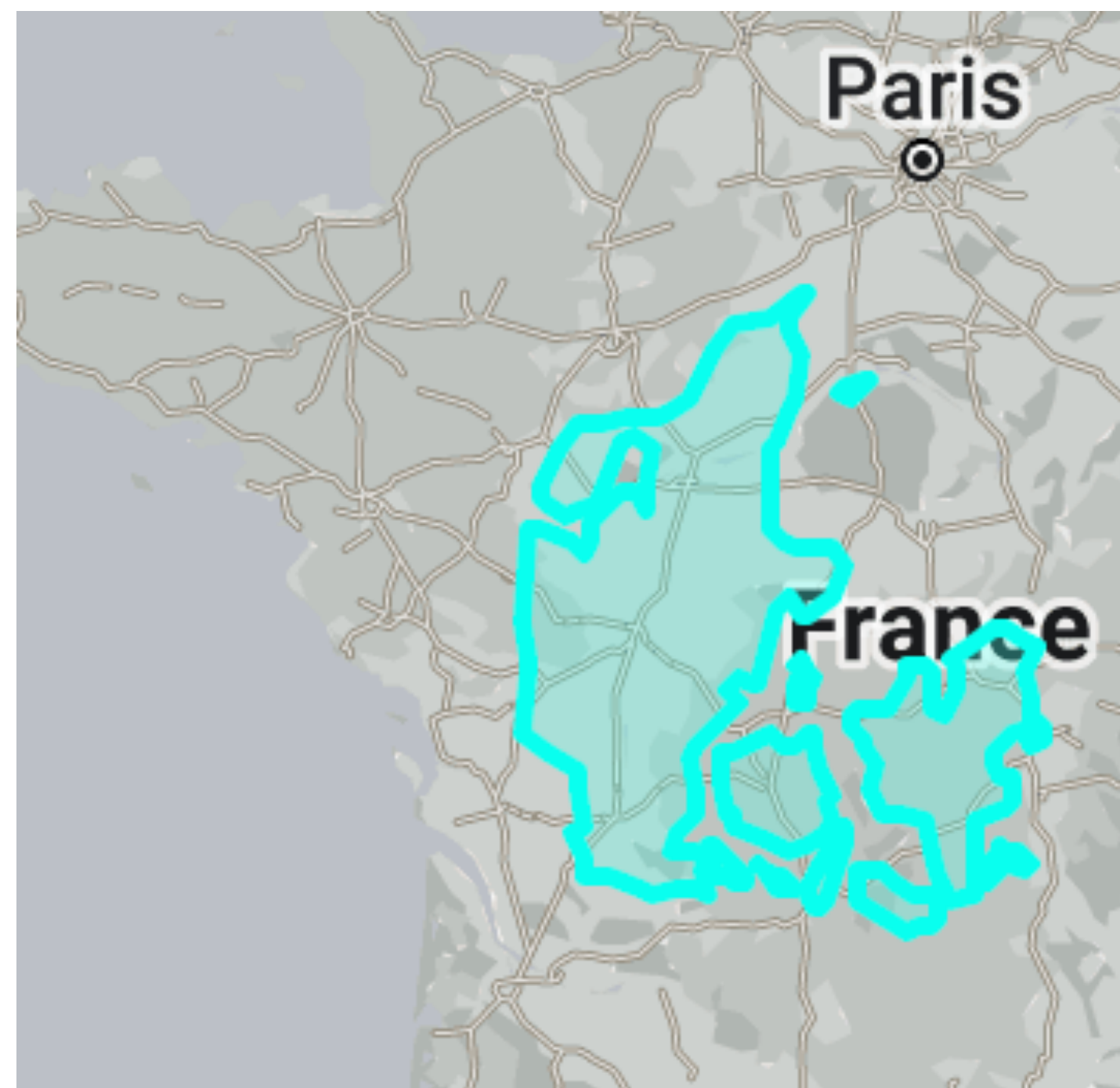
We identify map projections with their EPSG code

European Petroleum Survey Group



Debugging coordinate data

Check your CRS!



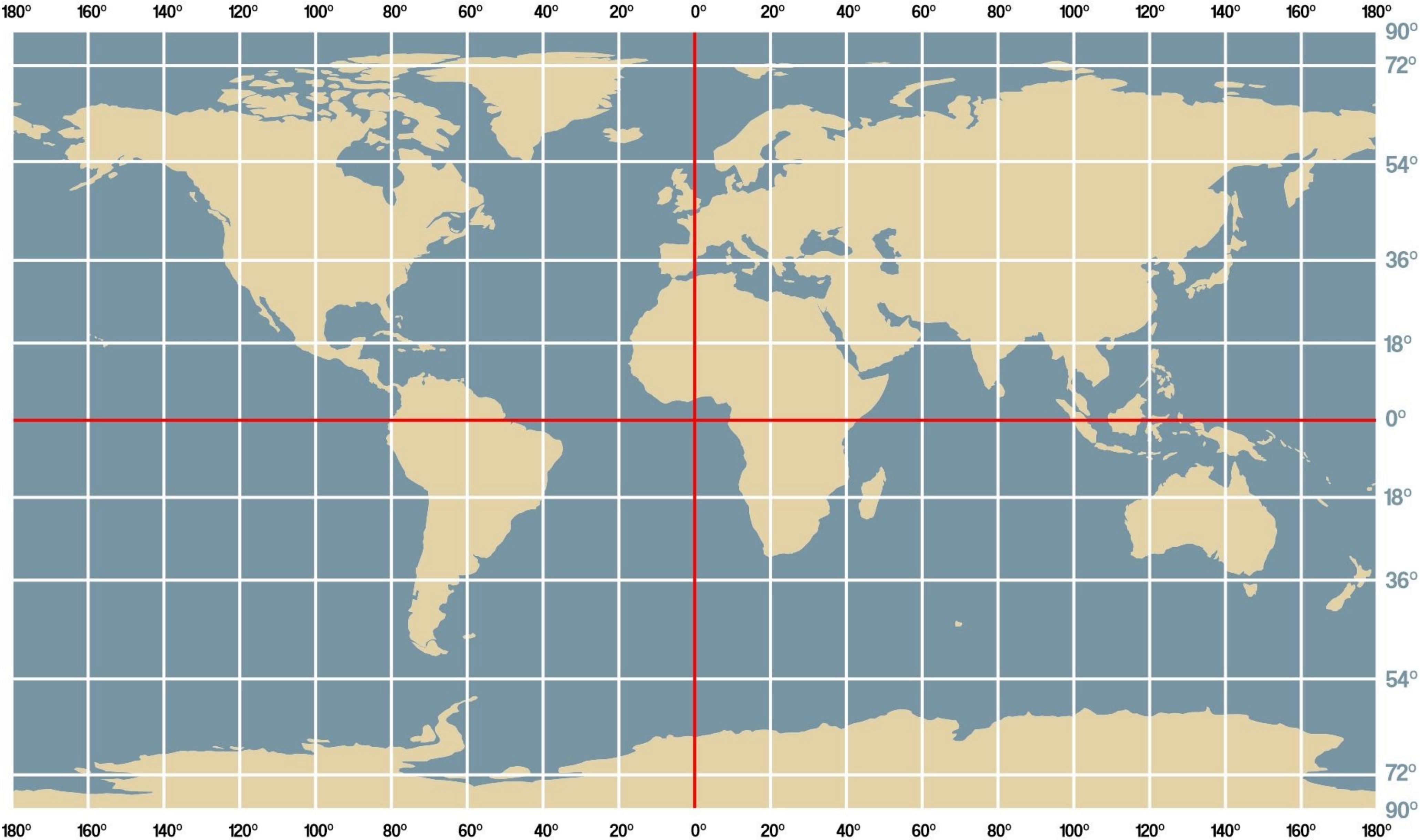
Debugging coordinate data

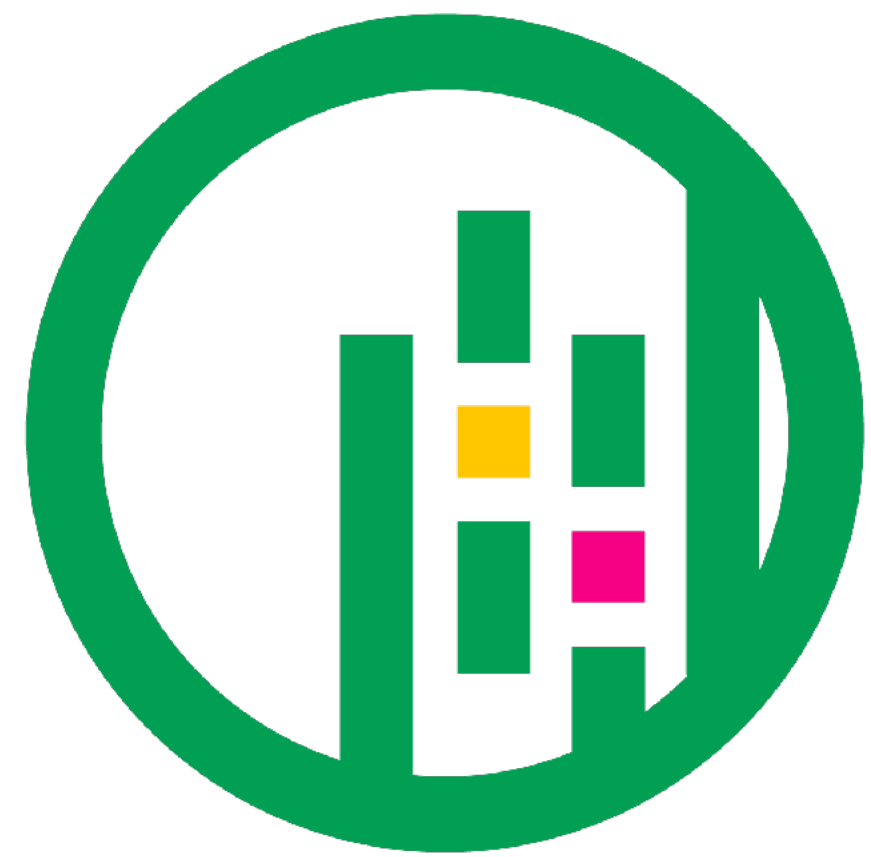
Are your data (x,y) or (y,x)?

lon, lat	lat, lon
formats • GeoJSON ref • KML ref • Shapefile ref • WKT ref • WKB ref • geobuf ref	formats • GeoRSS ref • Encoded Polylines (Google) ref • iCalendar ref
javascript apis • OpenLayers ref • d3 ref • ArcGIS API for JavaScript ref • Mapbox GL JS ref	javascript apis • Leaflet ref • Google Maps API ref

Beware of discontinuities, infinities, and pathologies

Blackboard





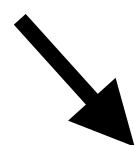
GeoPandas

The spatial extension of pandas

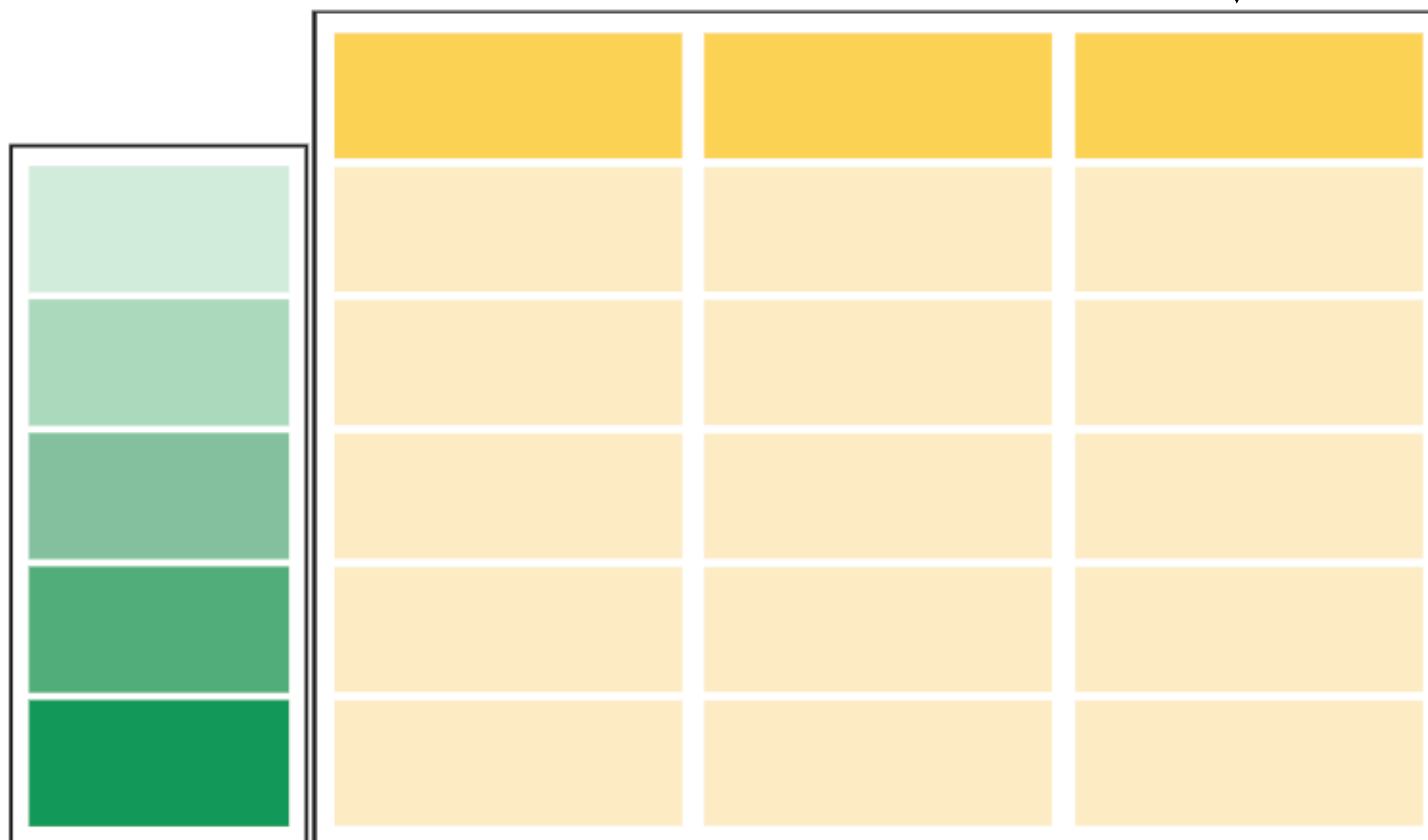
Pandas is a package for

data analysis

Dataframe



Series



index

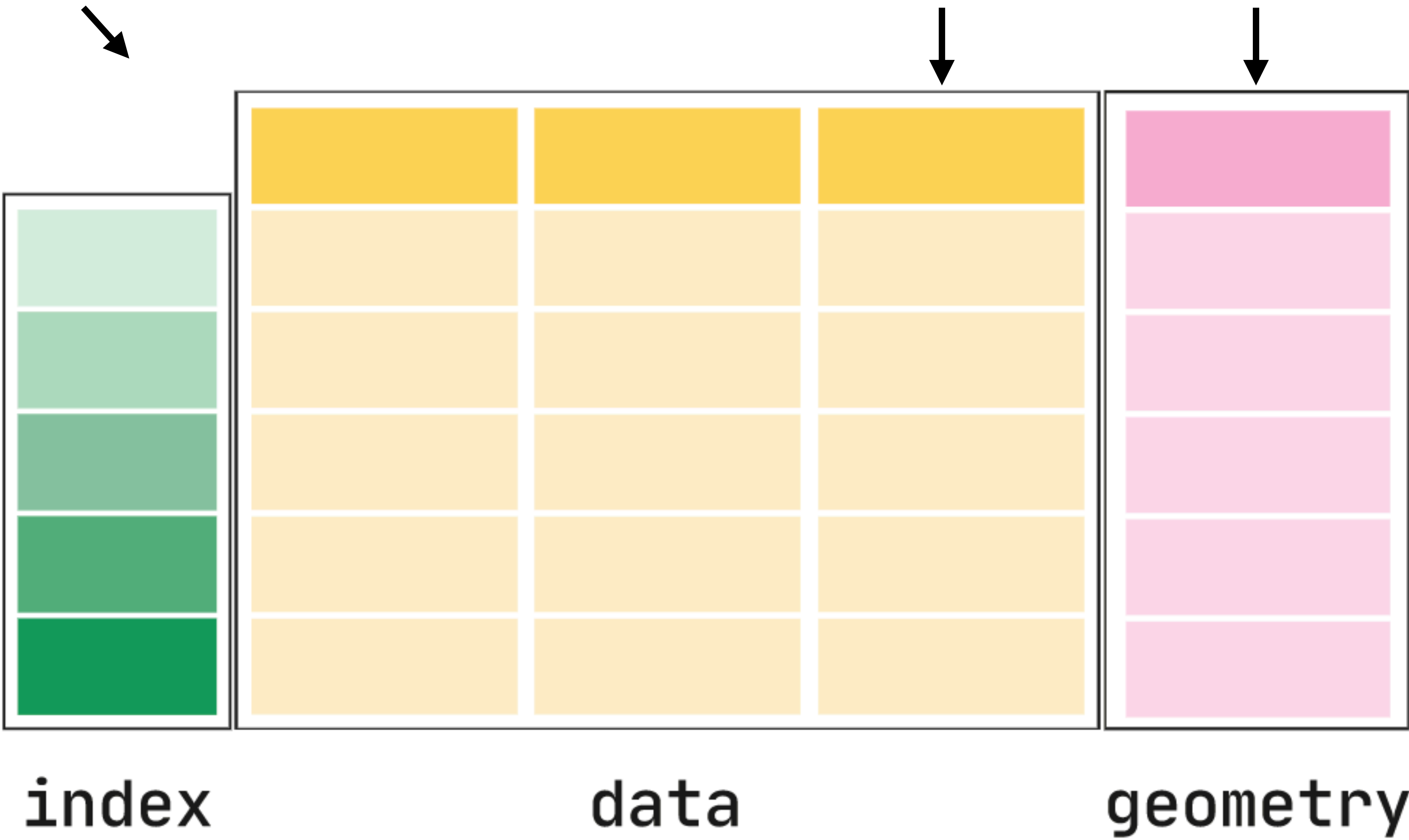
data

GeoPandas is a package for **geospatial** data analysis

GeoDataframe

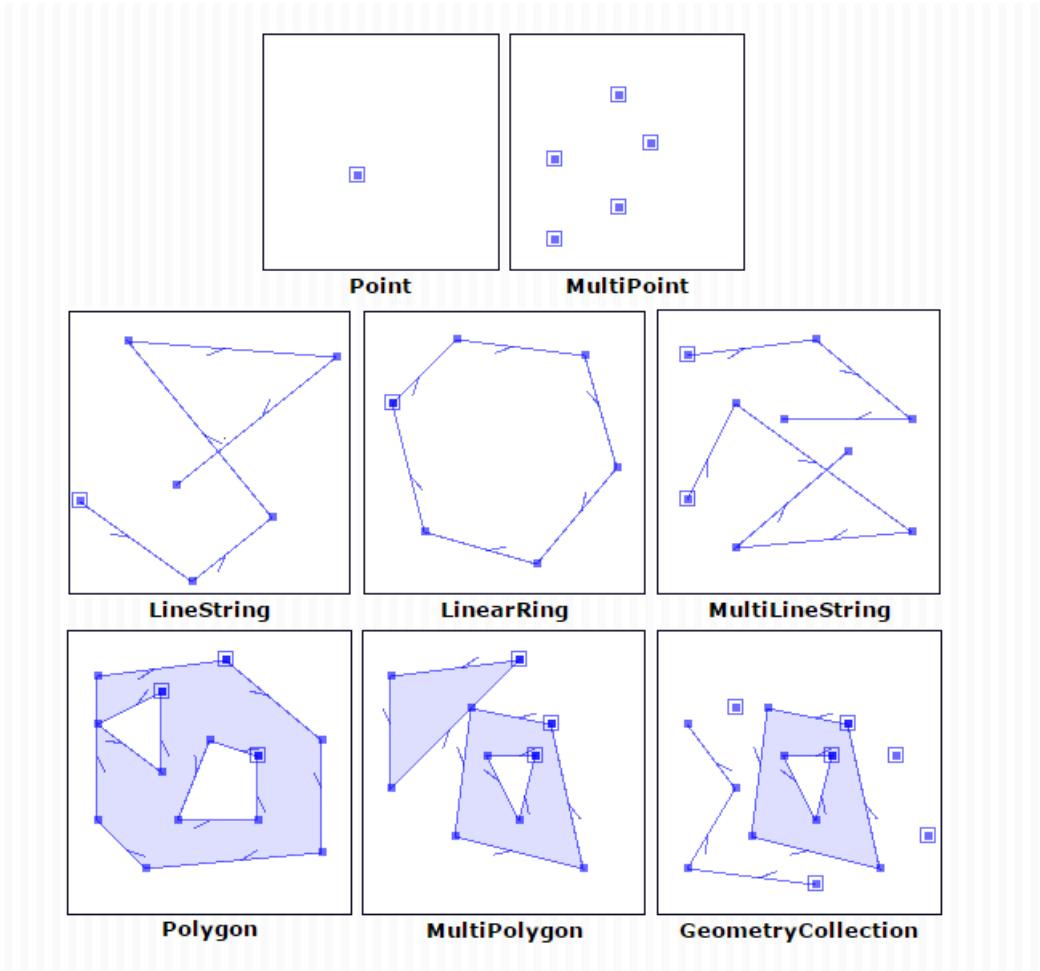
Series

GeoSeries



GeoPandas

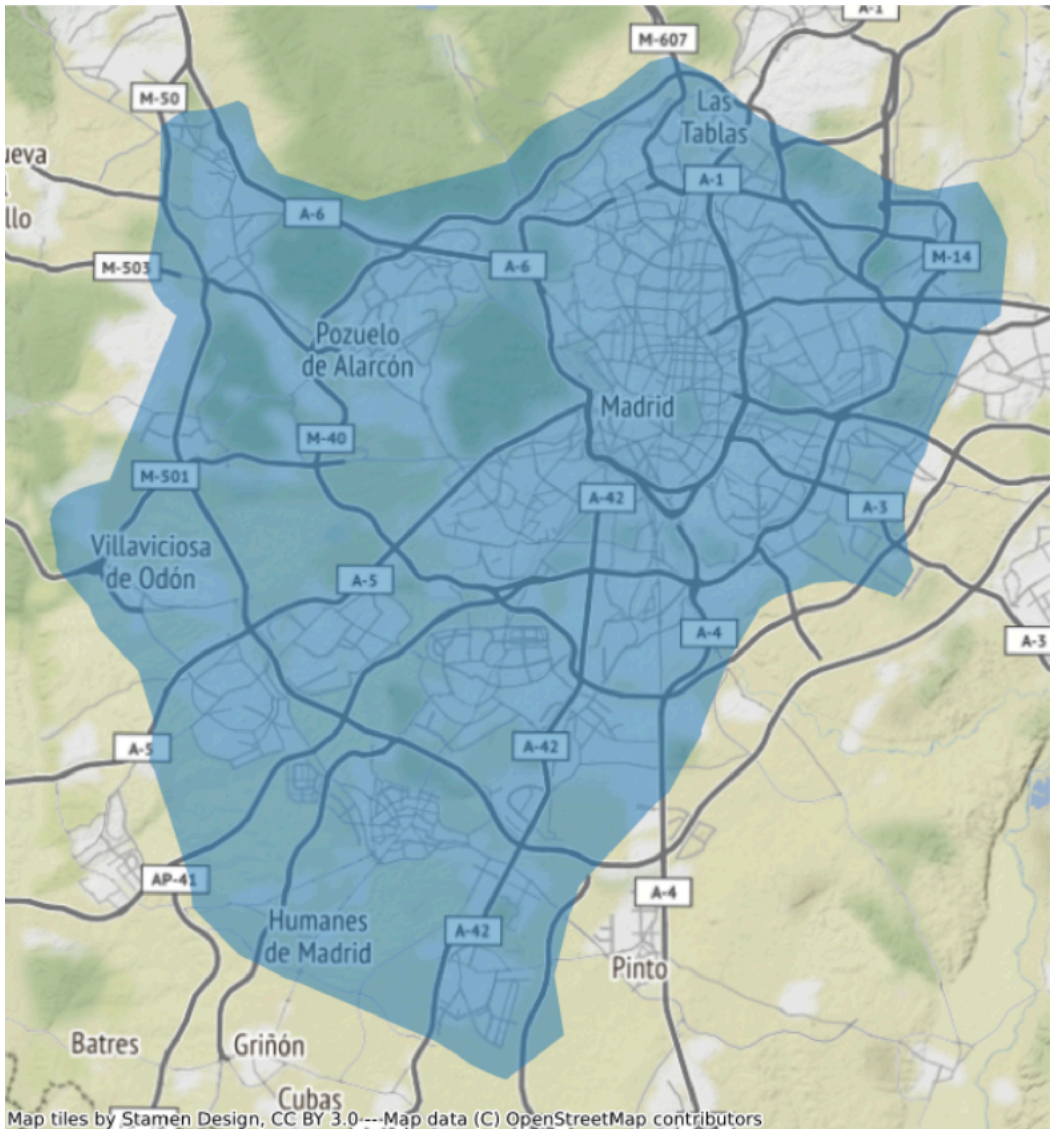
Uses Shapely for geometries



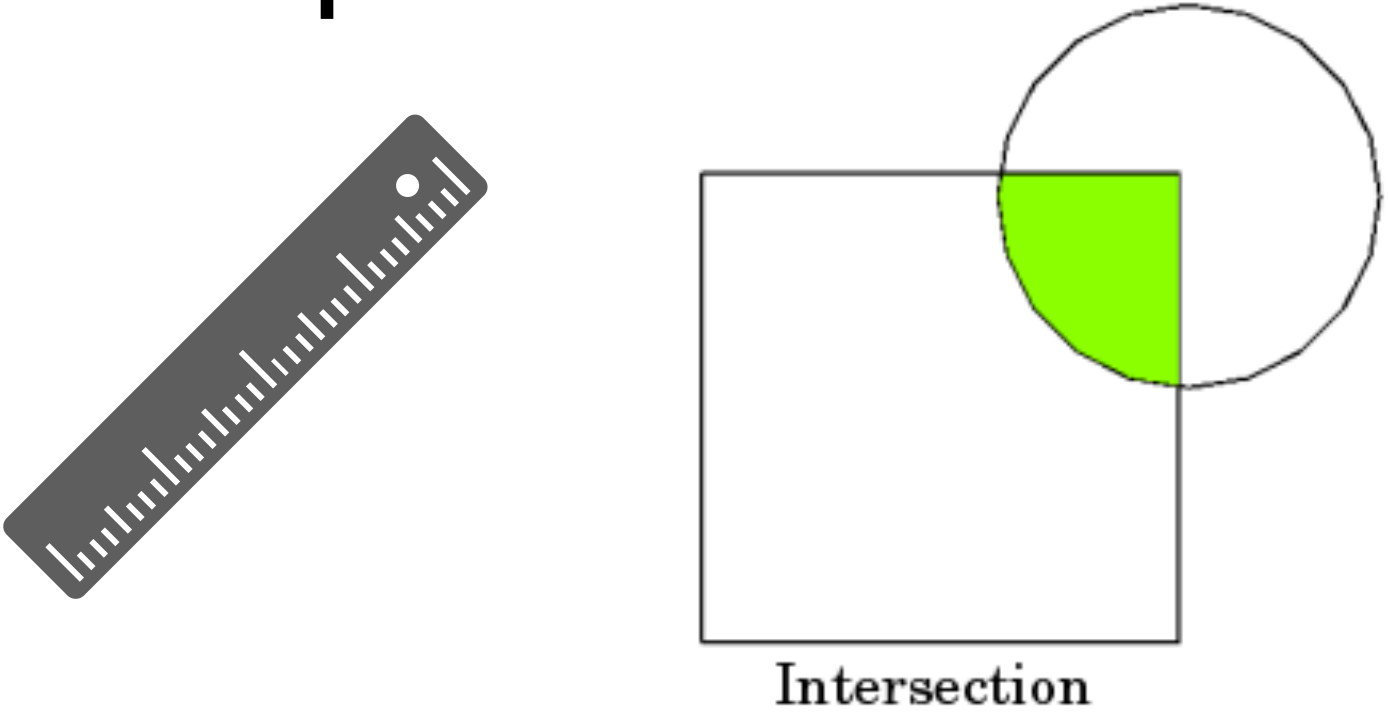
CRS support



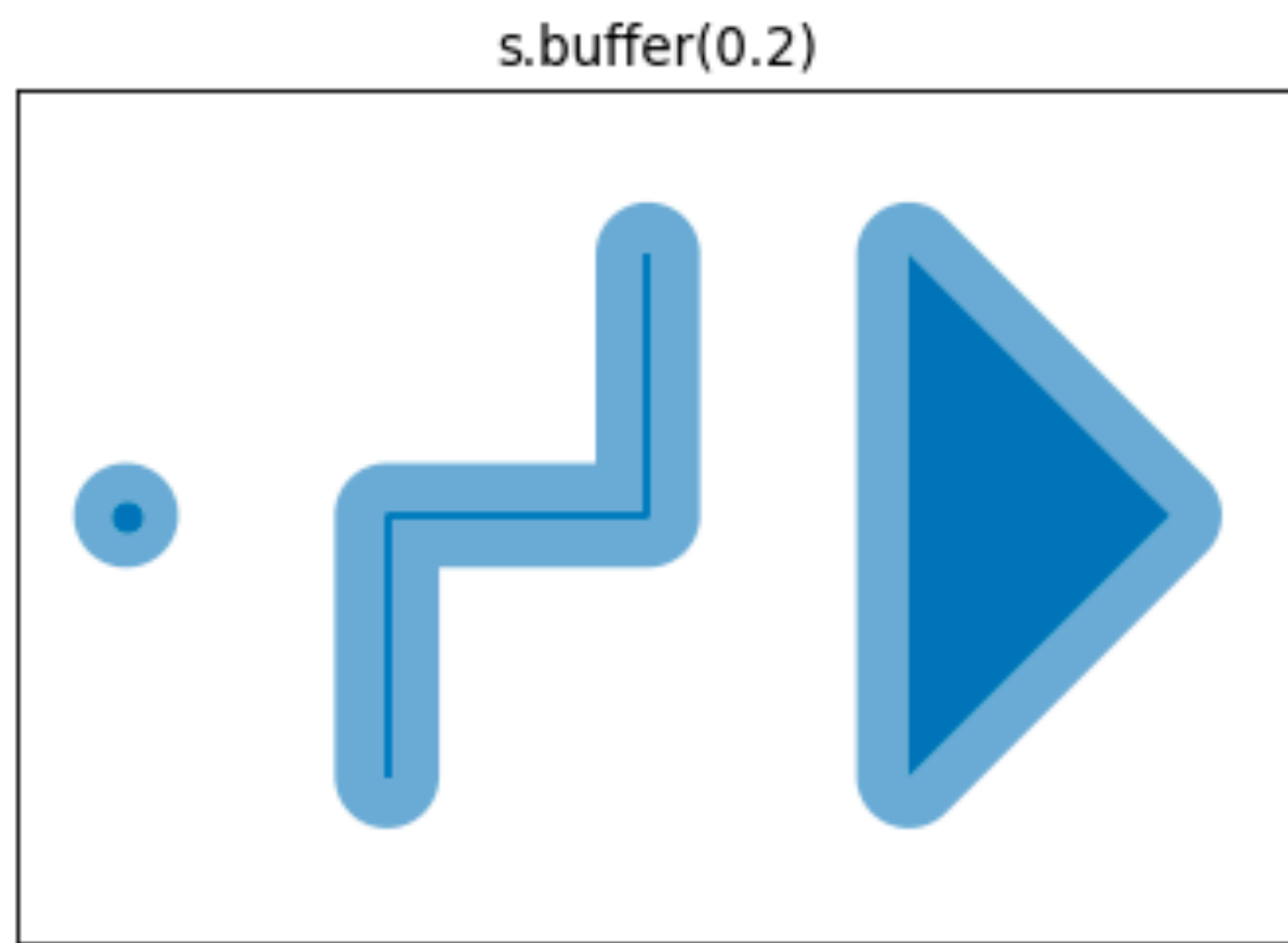
Static maps



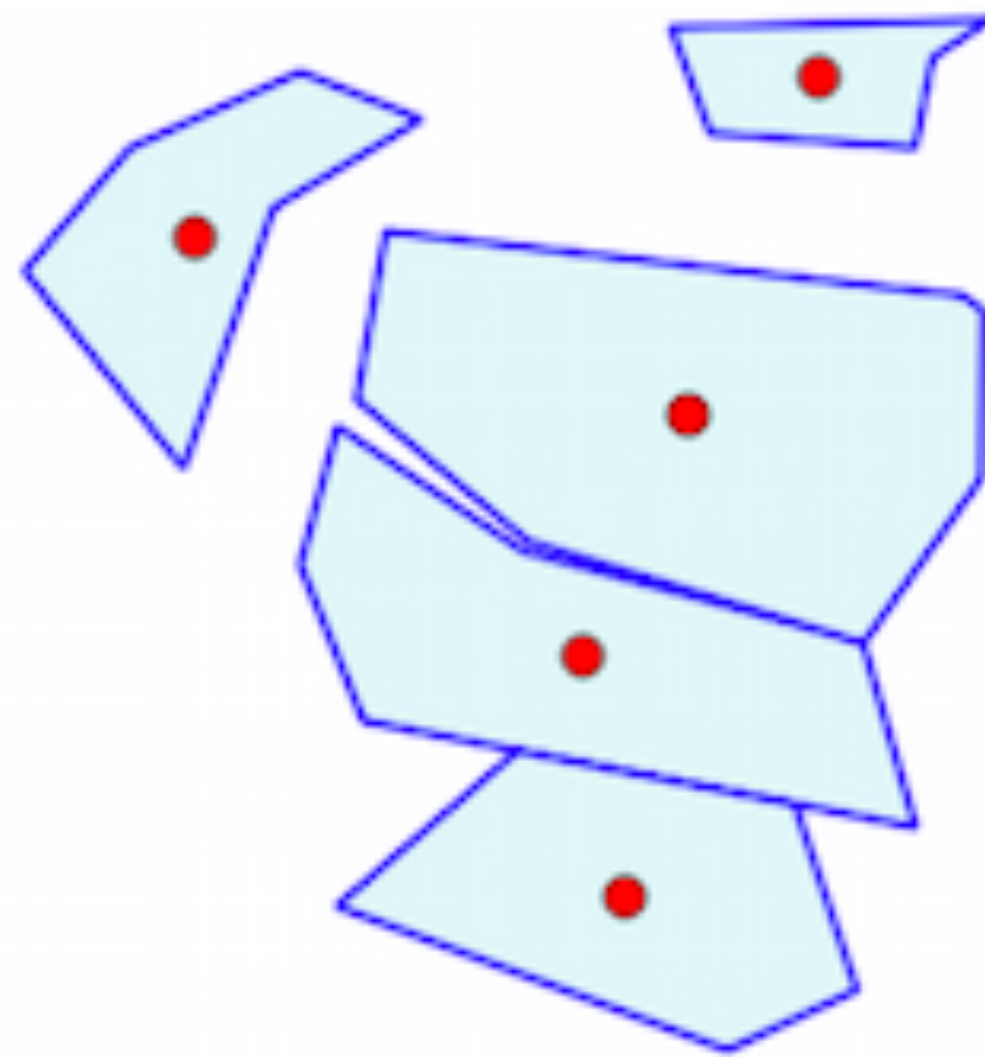
Spatial operations



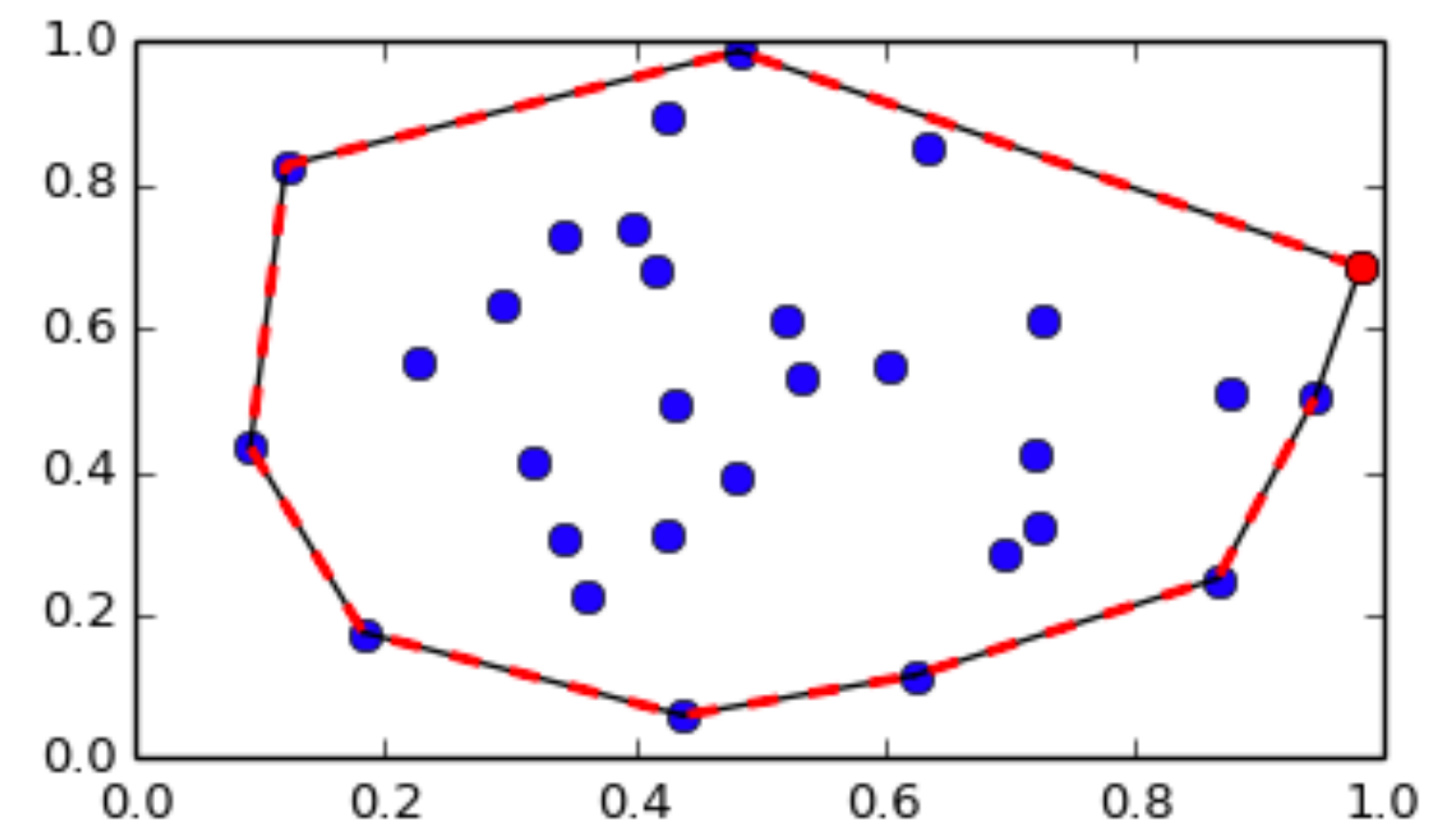
Geometric Manipulations



`gdf.buffer(10)`

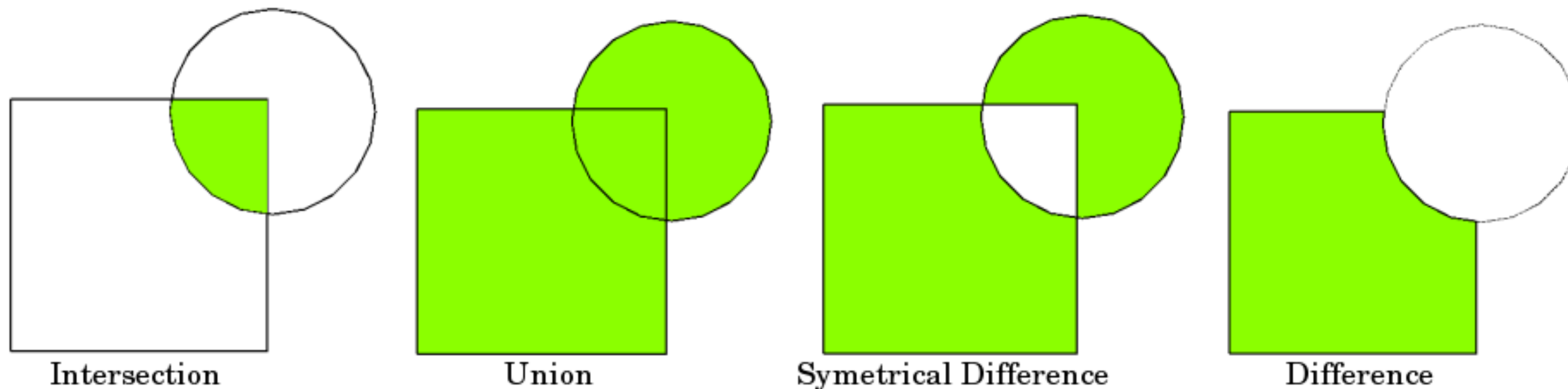


`gdf.centroid`



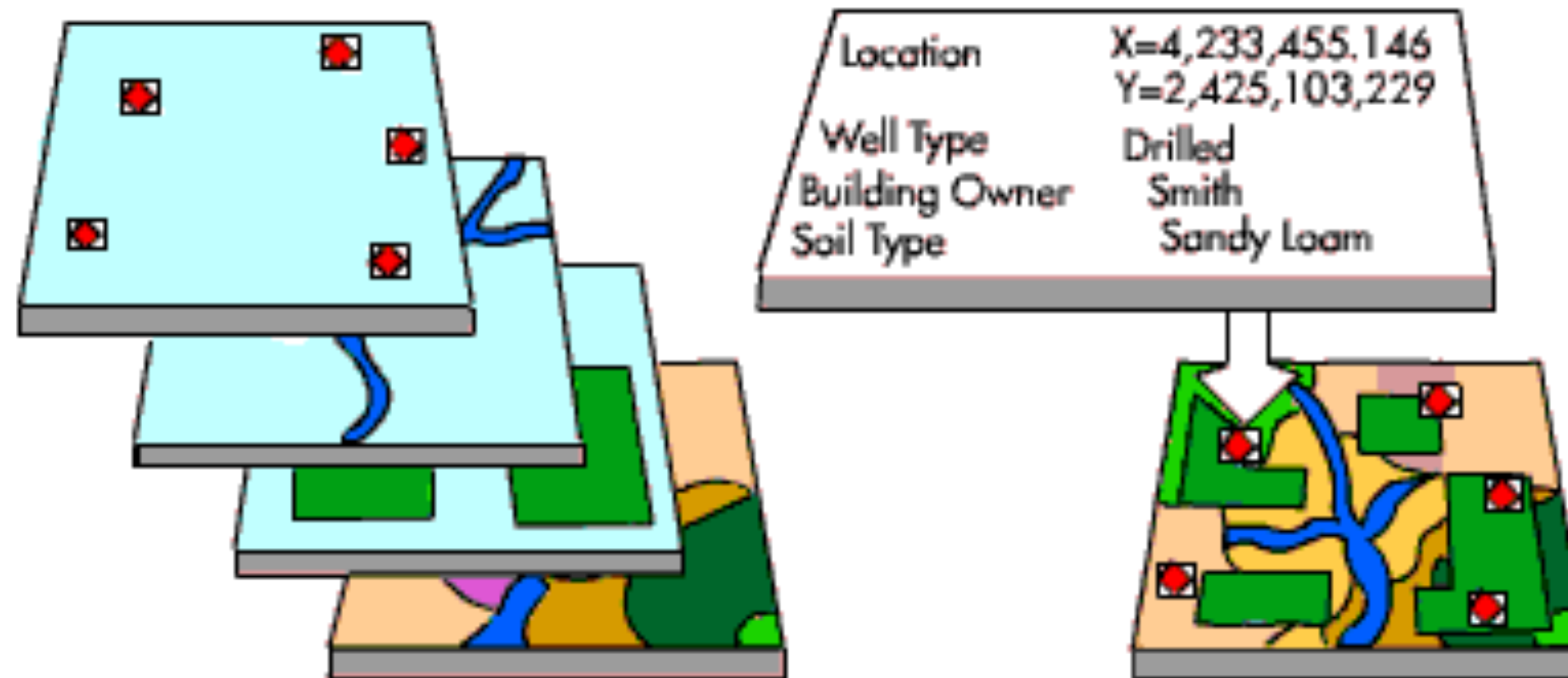
`gdf.convex_hull`

Set based operations



```
intersection = gdf1.overlay(gdf2, how='intersection')
```

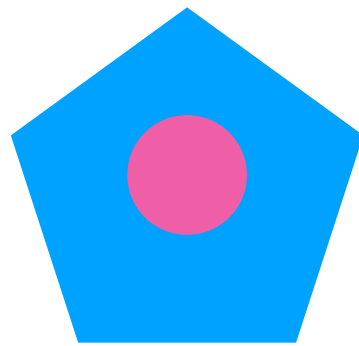

Spatial join



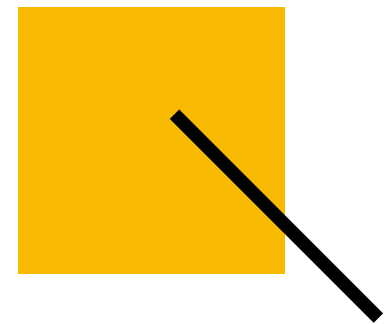
```
join = point_gdf.sjoin(poly_gdf)
```

Spatial queries

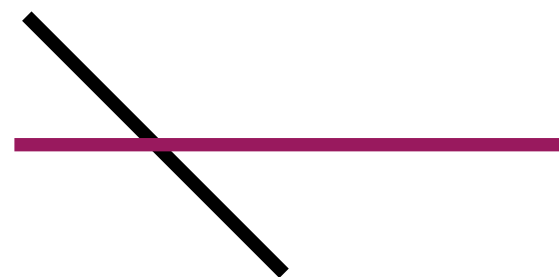
`point.within(poly)`



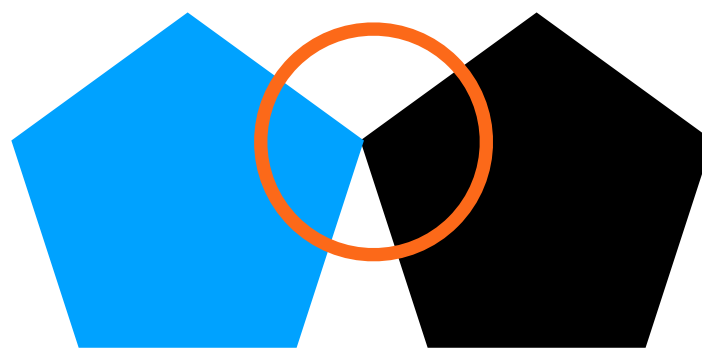
`line.intersects(poly)`



`line1.crosses(line2)`



`poly1.touches(poly2)`



`etc....`

Shapely+Geopandas have become much faster in 2024



geopandas v.0.*
shapely v.1.*



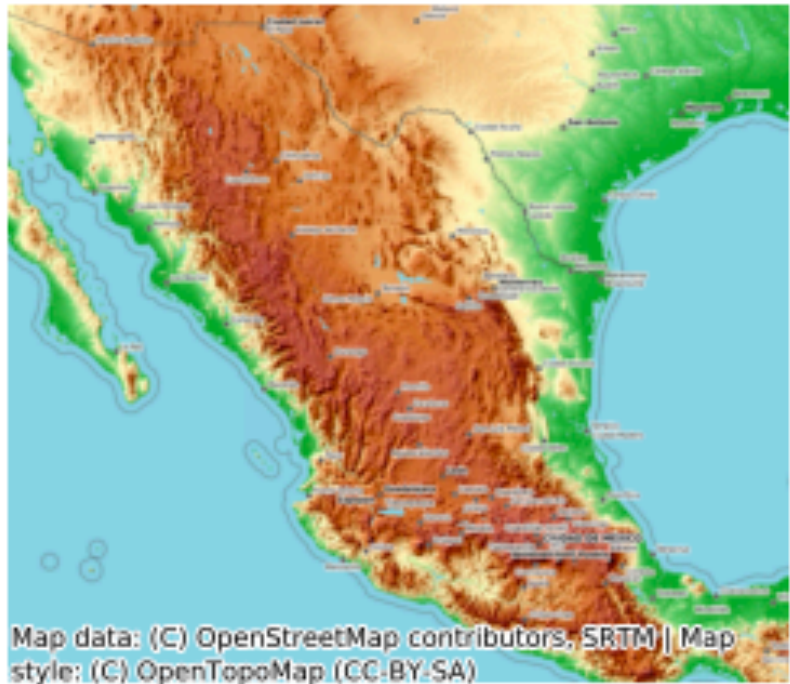
geopandas v.1.*
shapely v.2.*

Contextily plots a basemap of map tiles

OpenStreetMap.Mapnik



OpenTopoMap



Stamen.Toner



Stamen.TonerLite



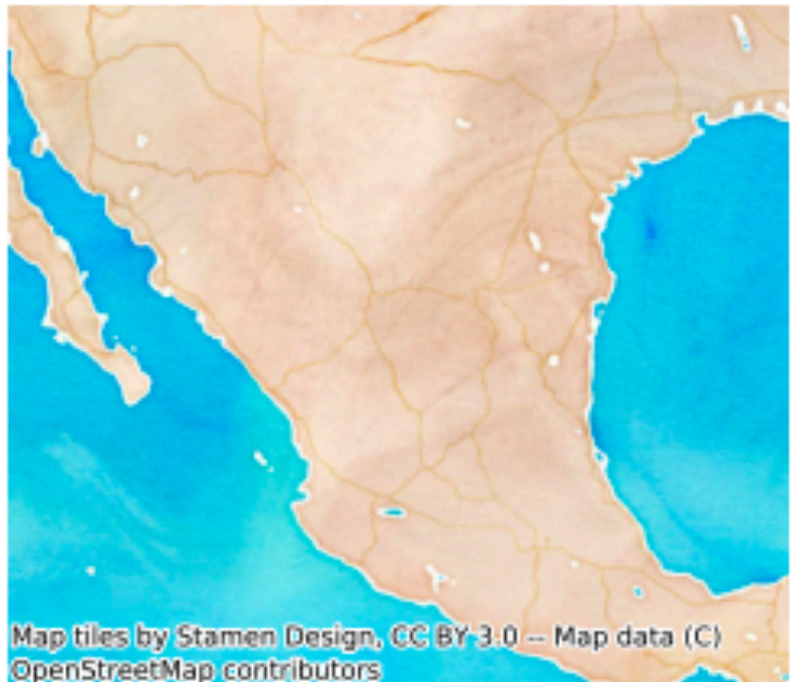
Stamen.Terrain



Stamen.TerrainBackground



Stamen.Watercolor



NASAGIBS.ViirsEarthAtNight2012



CartoDB.Positron



CartoDB.Voyager



Jupyter

We now know how to...

- **Read** vector data with *GeoPandas*
- Make simple **maps** with *matplotlib* and *contextily*
- Do **spatial operations** like buffering
- Modify the data's **CRS**

Next week: Choropleth maps & map classification

Denmark's coronavirus hotspots (by municipality), December 14th

Coronavirus cases per 100,000 residents over past 7 days as at December 14th (Source: SSI)

